



Study of Successive Over Relaxation (SOR) Method

*A Project Report Submitted in Partial Fulfillment of the Requirements for the
Degree of*

MASTER OF SCIENCE IN MATHEMATICS

by

ANUSHA

Register No: P05DG22S095031

under the supervision of

Dr. Ravisha M.

Designation

Dr. G. Shankar Government Women's First Grade College
and PG Study Centre, Ajjarkad, Udupi - 576101

submitted to

Department of Mathematics

Mangalore University

Mangalagangothri - 574 199

**Dr. G. Shankar Government Women's
First Grade College and PG Study Centre,
Ajjarkad, Udupi - 576101**

May 2020

Declaration

I, **ANUSHA** (Reg. No: **P05DG22S095031**) hereby declare that the Dissertation/Project Report entitled '**Study Of Successive Over Relaxation (SOR) Method**' submitted for the partial fulfilment of the M.Sc. Degree, is my collective original work and the Dissertation/Project Report has not formed the basis for the award of any degree, associateship, fellowship or any other similar purpose.

Place: Ajjarkad

Date:

Signature of the student

(**ANUSHA**)



**DR. G. SHANKAR GOVERNMENT WOMEN'S
FIRST GRADE COLLEGE AND PG STUDY CENTRE,
AJJARKAD, UDUPI - 576101**

CERTIFICATE

This is to certify that the dissertation entitled '**Study of Successive Over Relaxation (SOR) Method**' is the bonafide collective work carried out by Mis. **ANUSHA** (Reg. No: **P05DG22S095031**), IV Semester, M.Sc. (Mathematics), Department of Mathematics, Dr. G. Shankar Government Women's First Grade College and PG Study Centre, Ajjarkad, Udupi, during the academic year 2023-24, in partial fulfillment of the requirements for the award of the degree of Master of Science in Mathematics and the Dissertation/Project Report has not formed the basis for the award previously of any degree, diploma, associateship, fellowship or any other similar title.

Signature of the Supervisor with date
(Name of the Supervisor)

Signature of the Chairperson/HOD/Coordinator/Principal
(with date and seal)

Acknowledgements

I wish to express my sincere gratitude to Dr.Ravisha M for his guidance,inspiration and encouragement in carrying out this project work. I hereby take the opportunity to thank himfor his suggestion and support. I am also thankful to staff members of mathematics department ho rendered their help during the period of my project.Finally I want to thank my college,Dr. G. Shankar Government Women's First Grade College and PG Study Centre,Ajjarkadu Udupi.

I extend my sincere thanks to my family and friends for their timely help and constant support.

Anusha

Contents

Abstract	1
1 Introduction to Successive Over-Relaxation	2
1.1 Introduction	2
1.1.1 Convergence And Convergence Rate	4
1.2 Application of SOR	5
1.2.1 Some Applications of Successive Over Relaxation Method . .	9
1.2.2 Importance of Successive Over Relaxation method	11
2 Linear System of equations	13
2.1 Methods Of Solution	14
2.1.1 Matrix Inversion Method	17
2.1.2 Gauss Elimination Method	21
2.1.3 Gauss-Jordan Method	26
2.1.4 Cholesky's Triangularisation Method	30
2.1.5 Crout's Method	34
2.1.6 Jacobi's Iteration Method	38
2.1.7 Gauss-Seidal Iteration Method	42
2.1.8 SOR Is Better Than Other Methods	47
3 Successive Over Relaxation Method	50
3.0.1 Convergence of SOR Method	52
3.0.2 Advantages Of SOR	58
3.0.3 Disadvantages Of SOR	59
3.0.4 Types Of SOR	61
3.1 Conclusion	62
References	63

Abstract

The project explores the successive over-relaxation (SOR) method, focusing on its convergence properties, convergence rate, and applications in solving linear systems. It introduces SOR as an enhancement over traditional methods such as matrix inversion, Gauss elimination, and Gauss-Jordan elimination. By incorporating a relaxation factor, SOR accelerates the convergence process, making it more efficient for large, sparse systems commonly found in engineering and scientific computations.

In examining the convergence and convergence rate of the SOR method, the project delves into the selection of the optimal relaxation factor, ω , which is crucial for maximizing efficiency. Comparisons with Gauss elimination and Gauss-Jordan methods highlight SOR's superior performance in terms of iteration count and computational time. The project also demonstrates the application of SOR in practical scenarios, emphasizing its ability to handle large-scale problems more effectively than direct methods.

The project discusses the advantages of SOR, including faster convergence and reduced computational resources, alongside its disadvantages, such as the need for a suitable relaxation factor and potential difficulties in implementation. By evaluating SOR against other classical methods, the project provides a comprehensive understanding of why SOR is often preferred in numerical linear algebra, particularly for solving large, complex systems.

Chapter 1

Introduction to Successive Over-Relaxation

1.1 Introduction

A Successive Over-Relaxation (SOR) is one of the most important method for solution of large linear system equations. It has applications in Fluid Dynamics, mathematical programming, linear elasticity and machine learning etc. The examples of applications of SOR in Dynamics include study of steady heat conduction, turbulent flows, boundary layer flows or chemically reacting flows. For this reason, SOR method is important for both researchers and business policymakers. In the real world, time is always something valuable, something no one wants to waste; when it comes to solving systems of equations, it can sometimes be better to get a close approximation of the solution than to get the exact solution for this very reason, among others. This is where the successive over-relaxation method (SOR) can come into play. The industry

standard for finding exact methods, Gaussian elimination, requires approximately $O(n^3)$ operations to solve the system, which becomes time consuming when n gets big. SOR on the other hand, while only giving us an approximation, can give us these approximations much faster than Gaussian elimination can. SOR was developed in 1950 by David Young and H. Frankel in 1950 and was developed to be used on digital computers. It was developed by modifying the GaussSeidel iteration model.

Given a square system of n linear equations with unknown x : $Ax=b$, where :

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \quad X = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

Then A can be decomposed into a diagonal component D, and strictly lower and upper triangular components L and U:

$$A = D + L + U \text{ where}$$

$$D = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix} \quad L = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ a_{21} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & 0 \end{bmatrix} \quad U = \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}$$

The system of linear equations may be rewritten as:

$$(D + \omega L)x = \omega b - [\omega U + (\omega - 1)D]x$$

for a constant $\omega > 1$, called the relaxation factor.

The method of successive over-relaxation is an iterative technique that solves the left hand side of this expression for $\mathbf{x}$, using the previous value for $\mathbf{x}$ on the right hand side. Analytically, this may be written as:

$$x^{(k+1)} = (D + \omega L)^{(-1)}(\omega b) - [\omega U + (\omega - 1)D]x^{(k)} = L_{(\omega)}x^{(k)} + c,$$

where $\mathbf{x}^k$ is the k^{th} approximation of $\mathbf{x}$ and $\mathbf{x}^{k+1}$ is the next or $k + 1$ iteration of $\mathbf{x}$. However, by taking advantage of the triangular form of $(D + \omega L)$, the elements of $\mathbf{x}^{k+1}$ can be computed sequentially using forward substitution:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \left[\frac{\omega}{a_{ii}} \right] \left(b_i - \sum a_{ij}x_j^{k+1} - \sum a_{ij}x_j^k \right), \quad i=1,2,\dots,n$$

1.1.1 Convergence And Convergence Rate

The choice of relaxation factor ω is not necessarily easy, and depends upon the properties of the coefficient matrix. In 1947, Ostrowski proved that if A is symmetric and positive-definite then $\rho(L_\omega) < 1$ for $1 < \omega < 2$. Thus, convergence of the iteration process follows, but we are generally interested in faster convergence rather than just convergence.

The convergence rate for the SOR method can be analytically derived. One needs to assume the following

- ★ the relaxation parameter is appropriate: $\omega \in (0, 2)$
- ★ Jacobi's iteration matrix $C_{Jac} = I - D^{-1}A$ has only real eigenvalues
- ★ Jacobi's method is convergent $\mu = \rho(C_{Jac}) < 1$
- ★ the matrix decomposition $A = D + L + U$ satisfies the property that $\det(\lambda D + ZL + \frac{1}{z}U) = \det(\lambda D + L + U)$ for any $z \in \mathbb{C} \setminus (0)$ and $\lambda \in \mathbb{C}$

Then the convergence rate can be expressed as

$$\rho(C_\omega) = \begin{cases} \frac{1}{4}(\omega\mu + (\sqrt{\omega^2\mu^2 - (4\omega - 1)})^2), & 0 < \omega \leq \omega_{opt} \\ \omega - 1, & \omega < \omega_{opt} < 2 \end{cases}$$

where the optimal relaxation parameter is given by

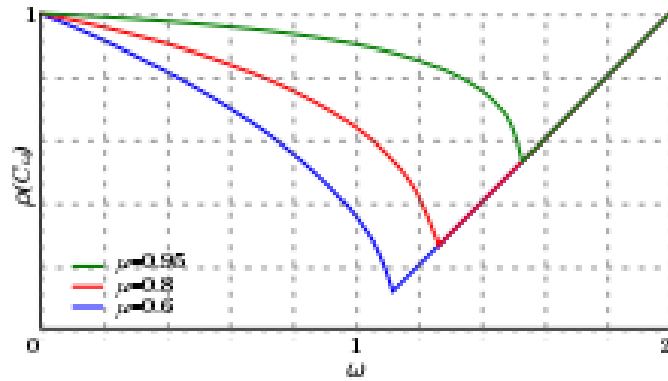
$$\omega_{opt} = 1 + \left(\frac{\mu}{1 + \sqrt{1 - \mu^2}} \right)^2 = 1 + \frac{\mu^2}{4} + O(\mu^3).$$

In particular, for $\omega=1$ (Gauss-Seidel) it holds that $\rho(C_\omega) = \mu^2 = \rho(C_{Jac}^2)$.

For the optimal ω we get $\rho(C_\omega) = \frac{1 - \sqrt{1 - \mu^2}}{1 + \sqrt{1 - \mu^2}} = \frac{\mu^2}{4} + O(\mu^3)$,

which shows SOR is roughly four times more efficient than Gauss-Seidel. The last assumption is satisfied for tridiagonal matrices since $Z(\lambda D + L + U)Z^{-1} = \lambda D + zL + \frac{1}{z}U$ for diagonal

Z with entries $Z_{ii} = z^{i-1}$ and $\det(\lambda D + L + U) = \det(Z(\lambda D + L + U)Z^{-1})$.



1.2 Application of SOR

Now let's move to our application of SOR. The application utilizes the heat equation, which is $\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} - \frac{\partial^2 u}{\partial z^2} \right) = 0$ Which simply states that as time increases, t , the temperature, u , and changes over the three special coordinates, xx, yy, zz where α is a positive constant. For our example, we're only going to use two dimensions, xx and yy . Let us move to that example. Utilizing an article by Ronal S. Besser we picked and modified his first simple example, the constant boundary temperature example. We are given a 0.9 x 0.9 m plate and its sides are heated to 273 Kelvin. See the model below for the full, initial set up.

273	273	273	273	273	273	273	273	273	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	0	0	0	0	0	0	0	0	273
273	273	273	273	273	273	273	273	273	273

The value at each entry in the matrix represents the temperature at that position, aka a node, on the plate. To solve this system, we used the finite difference method. The FDM utilizes some large but finite about of rectangular elements such that $u_{xxxx} + u_{yyyy} = 0$ Where u_{xxxx} and u_{yyyy} are the temperatures at the points x and y respectfully. We can rewrite the FDM equation We can rewrite the FDM equation like so

$$\frac{u_{i,j+1}}{(\Delta x^2)} - \left(\frac{2}{\Delta x^2} + \frac{2}{\Delta y^2} \right) u(i,j) + \left(\frac{1}{\Delta y^2} \right) u(i+1,j) + \left(\frac{1}{\Delta x^2} \right) u(i,j-1) + \left(\frac{1}{\Delta y^2} \right) u(i-1,j) = 0$$

and rewriting it and multiplying everything by -1, we get

$$\left(-\frac{1}{\Delta x^2} \right) u(i,j+1) + \left(\frac{2}{\Delta x^2} + \frac{2}{\Delta y^2} \right) u(i,j) + \left(\frac{1}{\Delta y^2} \right) u(i+1,j) = \left(\frac{1}{\Delta x^2} \right) u(i,j-1) + \left(\frac{1}{\Delta y^2} \right) u(i-1,j).$$

To set the matrix system up, first we need to establish out Δx and Δy values. For this, we set $\Delta x = \Delta y = 1$

$$\begin{bmatrix} 0.1118 \\ 0.1526 \\ 0.1613 \\ 0.1613 \\ 0.1569 \\ 0.1408 \\ 0.0847 \\ 0.0831 \end{bmatrix}$$

Running just the SOR through MATLAB, it takes 0.084 seconds from the setup of the matrix to displaying the results at the very end of it while MATLAB's built in functions can solve the exact answer in 0.020 seconds. While that is much faster, bear in mind SOR does not need to have the inverse of matrix A, which that alone can make a world of difference in deciding on which method to solve $AAAA = bb$. SOR can also be a very nice system to use if matrix A is very large and sparse.

Storing all of those zeros can be a waste of space in the computer as it slows down computation times. To set the matrix system up, first we need to establish out δxx and δyy values. For this, we set $\delta xx = \delta yy = 1$.

Our coefficient matrix A_2 , comes from the inner 8×8 matrix of zeros from the bigger matrix A , and b will be our vector of known values surrounding the nodal points of A_2 . To find our values, we start at the $A_{2,j}$ and add up the values around the node. If the value is known, it is put into the b vector, if it is unknown; it is set as a variable, x_1 through x_8 . As an example, to find the first row of A_2 , we add the 273 at position $A_{2,1}$ and the 273 at $A_{1,2}$ from the larger matrix A . Those values are stored as b_1 . $A_{2,1}$ is treated as $4 * x_1$ based on our choices in delta x and delta y $A_{2,3}$ is $-1 * x_2$ $A_{2,1}$ is $-1 * x_1$ in the row below. Our full system looks like this.

$$\begin{bmatrix} 4 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 4 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 4 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 4 \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \end{bmatrix} = \begin{bmatrix} 546 \\ 273 \\ 273 \\ 273 \\ 273 \\ 273 \\ 273 \\ 546 \end{bmatrix}$$

Before we move on to the coding of the problem into MATLAB, let's first discuss how the relaxation factor was chosen. If A is a positive definite matrix and $0 < \omega < 2$, then the SOR is guaranteed to converge for any initial choice of ω . If in addition A is tridiagonal and $\rho(t_g)$ where $T_g = D^{-1}(L + U) < 1$ then the optimal choice of the relaxation factor is $\omega = \frac{2}{1 + \sqrt{1 - |\rho(t_g)|^2}}$ which is how we obtained our optimum relaxation factor, 1.1805. All of our pieces are now set, let's put it all together in MATLAB. Running the code through, the x vector's last iteration is

$$\begin{bmatrix} 192.4278 \\ 168.6481 \\ 161.6298 \\ 159.5585 \\ 158.9471 \\ 158.7667 \\ 158.7135 \\ 158.6978 \end{bmatrix}.$$

If we compare that with the exact results of $\text{inv}(A) * b$ we get

$$\begin{bmatrix} 173.0784 \\ 146.3137 \\ 139.1765 \\ 137.3922 \\ 137.9222 \\ 139.1765 \\ 146.3137 \\ 173.0784 \end{bmatrix}$$

and if we compare the errors as decimals, we get

$$\begin{bmatrix} 0.1118 \\ 0.1526 \\ 0.1613 \\ 0.1613 \\ 0.1569 \\ 0.1408 \\ 0.0847 \\ -0.0831 \end{bmatrix}.$$

Compare the times. Running just the SOR through MATLAB, it takes 0.084 seconds from the setup of the matrix to displaying the results at the very end of it while MATLAB's built in functions can solve the exact answer in 0.020 seconds. While that is much faster, bear in mind SOR does not need to have the inverse of matrix A, which that alone can make a world of difference in deciding on which method to solve . SOR can also be a very nice system to use if matrix A is very large and sparse. Storing all of those zeros can be a waste of space in the computer as it slows down computation times.

Here , we have highlighted the importance of using SOR interactive solve method for accelerating solution of real word problems.It should come as no surprise the SOR method is an industry standard for solving matrix systems . Further research will be needed to find an SOR algorithm that would produce better, closer results to the exact values. That being said, where the SOR method shines through lays in its speed and its ability to solve any $n \times n$ system without the need of the coefficient matrix A having an inverse and without the need to store the matrix entirely, saving space and time.

1.2.1 Some Applications of Successive Over Relaxation Method

Successive Over Relaxation (SOR) method is a powerful numerical technique used primarily to solve linear systems of equations. It's a variant of the Gauss-Seidel method and is particularly useful for solving large systems of linear equations iteratively. Here are some applications where the SOR method finds utility:

- 1. Finite Element Analysis (FEA):** FEA involves discretizing a continuous system into a finite number of elements. SOR can be employed to solve the resulting large sparse linear systems arising from the discretization process.

- 2. Electromagnetic Field Simulation :** In computational electromagnetics, Maxwell's equations can be discretized using finite-difference or finite-element methods, resulting in large linear systems that can be efficiently solved using SOR. Fluid Dynamics: Computational fluid dynamics (CFD) simulations often involve solving linear systems derived from the discretization of the Navier-Stokes equations. SOR can be used as an iterative solver in these simulations.

3. Structural Analysis : SOR can be applied to solve linear systems arising in structural analysis problems, such as finite element modeling of mechanical structures or stress analysis.

4. Heat Transfer : When modeling heat transfer problems using numerical methods like finite differences or finite elements, SOR can be used to solve the resulting linear systems arising from the discretization of the heat conduction equation.

5. Quantum Mechanics : In computational quantum mechanics, particularly in solving the Schrödinger equation for multi-electron systems, discretization methods often lead to large linear systems that can be efficiently solved using iterative methods like SOR.

6. Image Processing : SOR can also be used in certain image processing applications, such as solving large linear systems that arise in image reconstruction problems, like those encountered in tomography or MRI reconstruction. Optimization Problems: SOR can be used as a solver for certain optimization problems, particularly those that can be formulated as linear systems. Overall, the SOR method is valuable in a wide range of scientific and engineering disciplines where large linear systems need to be solved efficiently. Its iterative nature and ability to handle large sparse matrices make it particularly well-suited for these applications.

7. Financial Modeling : SOR can be employed in financial modeling and computational finance to solve linear systems arising from pricing derivatives, risk management, portfolio optimization, and other financial analyses. It enables efficient and accurate numerical solutions to complex financial models and equations.

8. Boundary Value Problems : SOR can be used to solve boundary value problems in physics and engineering, where the solution must satisfy specified conditions at the boundaries of the problem domain. It helps in approximating the solutions to differential equations subject to boundary conditions, such as

Poisson's equation and Laplace's equation.

These are just a few examples of how the successive over-relaxation method finds application across various fields, showcasing its versatility and effectiveness in solving large-scale linear systems encountered in computational science and engineering.

1.2.2 Importance of Successive Over Relaxation method

The successive over-relaxation (SOR) method holds significant importance in various fields of science and engineering for several reasons:

1 Efficiency : SOR often converges faster than the standard Gauss-Seidel method or Jacobi method for solving large systems of linear equations. Its ability to accelerate convergence makes it an attractive choice for iterative solution techniques, particularly when dealing with computationally intensive problems.

2. Scalability : SOR is well-suited for large-scale problems encountered in computational science and engineering. It efficiently handles systems with a large number of equations and unknowns, making it applicable to simulations and analyses involving complex physical phenomena and intricate mathematical models.

3. Robustness : SOR exhibits robust convergence properties across a wide range of problem types and parameter configurations. It can handle systems with various characteristics, such as symmetric, diagonally dominant, or indefinite matrices, without sacrificing convergence behavior or stability.

4. Versatility : SOR is versatile and adaptable to different problem domains and applications. It finds utility in diverse fields, including computational fluid dynamics, structural analysis, electromagnetics, optimization, image processing, and financial modeling, demonstrating its effectiveness in addressing various scientific and engineering challenges.

5. Numerical Stability : SOR maintains numerical stability during the iterative process, ensuring that the solution remains accurate and reliable even

when dealing with ill-conditioned matrices or noisy data. Its stability properties make it suitable for solving linear systems encountered in real-world scenarios with inherent uncertainties.

6. Convergence Control : SOR allows for tuning the relaxation parameter (often denoted as ω) to control the rate of convergence and stability of the iterative process. Adjusting the relaxation parameter enables users to optimize convergence speed while maintaining stability, offering flexibility in solving different problem instances.

7. Parallelization : SOR can be parallelized efficiently, enabling distributed computing and leveraging parallel architectures to accelerate convergence and reduce computational time for large-scale simulations. Parallel implementations of SOR further enhance its applicability to high-performance computing environments.

8. Foundation for Advanced Methods : SOR serves as the basis for more sophisticated iterative techniques, such as multigrid methods and preconditioned iterative solvers. By understanding the principles and techniques underlying SOR, researchers and practitioners can develop and enhance advanced algorithms for solving complex numerical problems.

Overall, the importance of the successive over-relaxation method lies in its efficiency, scalability, robustness, versatility, numerical stability, convergence control, parallelizability, and role as a foundational technique for advanced iterative solvers, making it a valuable tool in computational science and engineering.

Chapter 2

Linear System of equations

In this chapter we present the solution of n linear simultaneous algebraic equations in n unknowns. Linear systems of equations are associated with many problems in engineering and science, as well as with applications of mathematics to the social sciences and quantitative study of business and economic problems. A system of algebraic equations has the form

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n\end{aligned}$$

where the coefficients a_{ij} and the constants b_j are known and x_i represents the unknowns. In matrix notation, the equations are written as

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix} \quad (2.1a)$$

$$\text{or simply } \mathbf{Ax} = \mathbf{b} \quad (2.1b)$$

A system of linear equations in n unknowns has a unique solution, provided that the determinant of the coefficient matrix is non-singular i.e., if $|A| \neq 0$. The rows and columns of a non-singular matrix are linearly independent in the sense that no row (or column) is a linear combination of the other rows (or columns): If the coefficient matrix is singular, the equations may have infinite number of solutions, or no solutions at all, depending on the constant vector. Linear algebraic equations occur in almost all branches of engineering. Their most important application

in engineering is in the analysis of linear systems (any system whose response is proportional to the input is deemed to be linear). Linear systems include structures, elastic solids, heat flow, seepage of fluids, electromagnetic fields and electric circuits i.e., most topics taught in an engineering curriculum. If the system is discrete, such as a truss or an electric circuit, then its analysis leads directly to linear algebraic equations.

Summarising, the modelling of linear systems invariably gives rise to equations of the form $Ax = b$, where b is the input and x represents the response of the system. The coefficient matrix A , which reflects the characteristics of the system, is independent of the input. In other words, if the input is changed, the equations have to be solved again with a different b , but the same A . Hence, it is desirable to have an equation solving algorithm that can handle any number of constant vectors with minimal computational effort.

2.1 Methods Of Solution

There are two classes of methods for solving system of linear, algebraic equations: direct and iterative methods. The common characteristics of direct methods are that they transform the original equation into equivalent equations (equations that have the same solution) that can be solved more easily. The transformation is carried out by applying certain operations.

The solution does not contain any truncation errors but the round off errors is introduced due to floating point operations.

Iterative or indirect methods, start with a guess of the solution x , and then repeatedly refine the solution until a certain convergence criterion is reached. Iterative methods are generally less efficient than direct methods due to the large number of operations or iterations required.

Iterative procedures are self-correcting, meaning that round off errors (or even arithmetic mistakes) in one iteration cycle are corrected in subsequent cycles. The solution contains truncation error. A serious drawback of iterative methods is that they do not always converge to the solution. The initial guess affects only the number of iterations that are required for convergence. The indirect solution technique (iterative) is more useful to solve a set of ill-conditioned equations

In this chapter, we will learn **methods** and **indirect(iterative)methods**.

Direct Methods :

★ inversion of a matrix

1. Matrix Inverse Method
2. Cramer's Method
3. Gauss Elimination Method
4. Gauss-Jordan Method
5. Cholesky's Triangularisation Method
6. Crout's Method

Indirect or Iterative Methods :

1. Jacobi's Iteration Method
2. Gauss-Seidal Iteration Method

★ The Inverse Of a Matrix

If $\mathbf{A}$ and $\mathbf{B}$ are $m \times n$ matrices such that

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I} \quad (2.2)$$

then $\mathbf{B}$ is said to be the inverse of $\mathbf{A}$ and is denoted by

$$\mathbf{B} = \mathbf{A}^{-1} \quad (2.2a)$$

In order to find the inverse $\mathbf{A}^{-1}$, provided the matrix $\mathbf{A}$ is given, let us consider the product

$$\mathbf{A} \text{adj} \mathbf{A} =$$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \times \begin{bmatrix} |M_{11}| & -|M_{21}| & \cdots & (-1)^{1+n}|M_n| \\ -|M_{21}| & |M_{22}| & \cdots & (-1)^{2+n}|M_n| \\ \cdots & \cdots & \cdots & \cdots \\ (-1)^{1+n}|M_{1n}| & (-1)^{2+n}|M_{2n}| & \cdots & |M_{nn}| \end{bmatrix}$$
$$= \left[\sum_{j=1}^n (-1)^{i+j} a_{kj} |M_{ij}| \right] \quad (2.3)$$

An element of the matrix on the right side of Eq. (2.3) has the value

$$\left[\sum_{j=1}^n (-1)^{i+j} a_{kj} |M_{ij}| \right] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} = |A| \text{ if } i = k \quad (2.4)$$

If $i \neq k$ the determinant possesses two identical rows, since the determinant corresponding to $i \neq k$ is obtained from the matrix $[a]$ by replacing the i^{th} row by the k^{th} row and keeping the k th row intact. Therefore, if $i \neq k$ the value of the element is zero. Equation (2.3) can be written as

$$\mathbf{A} \text{Adj} \mathbf{A} = |\mathbf{A}| \mathbf{I} \quad (2.5)$$

Premultiplying Eq. (2.5) throughout by $\mathbf{A}^{-1}$ and dividing the result by $|\mathbf{A}|$, we get

$$\mathbf{A}^{-1} = \frac{\text{adj} \mathbf{A}}{\det \mathbf{A}}$$

so that the inverse of a matrix $\mathbf{A}$ is obtained by dividing its adjoint matrix by its determinant $|\mathbf{A}|$. If $\det \mathbf{A}$ is equal to zero, then the elements of $\mathbf{A}^{-1}$ approach infinity (or are indeterminate at best), in which case the inverse $\mathbf{A}^{-1}$

is said not to exist, and the matrix $\mathbf{A}$ is said to be singular. The inverse of a matrix exists only if determinant is not zero, that is, the matrix must be non-singular. There is no direct division of matrices. The operation of division is performed by inversion; if

$$\mathbf{AB} = \mathbf{C}$$

$$\text{then } \mathbf{B} = \mathbf{A}^{-1} \mathbf{C}$$

where $\mathbf{A}^{-1}$ is called the inverse of matrix $\mathbf{A}$.

The requirements for obtaining a unique inverse of a matrix are: 1. The matrix is a square matrix

2. The determinant of the matrix is not zero (the matrix is non-singular) The inverse of a matrix is also defined by the relationship:

$$\mathbf{A}^{-1} \mathbf{A} = \mathbf{I}$$

The following are the properties of an inverted matrix:

1. The inverse of a matrix is unique.
2. The inverse of the product of two matrices is equal to the product of the inverse of the two matrices in reverse order: $(\mathbf{AB})^{-1} = \mathbf{B}^{-1} \mathbf{A}^{-1}$
3. The inverse of a triangular matrix is itself a triangular matrix of the same type.

4. The inverse of a symmetrical matrix is itself a symmetrical matrix.
5. The negative powers of a non-singular matrix are obtained by raising the inverse of the matrix to positive powers.
6. The inverse of the transpose of A is equal to the transpose of the inverse of A
 $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Example :

Find the inverse of the matrix $\mathbf{A} = \begin{bmatrix} 2 & 3 \\ 5 & 1 \end{bmatrix}$

Solution :

If $\mathbf{A} = \begin{bmatrix} 2 & 3 \\ 5 & 1 \end{bmatrix}$

Then, $\text{adj } \mathbf{A} = \begin{bmatrix} 1 & -3 \\ -5 & 2 \end{bmatrix}$

and $\det \mathbf{A} = 2 \times 1 - 5 \times 3 = -13$

Hence $\mathbf{A}^{-1} = -\frac{1}{13} \begin{bmatrix} 1 & -3 \\ -5 & 2 \end{bmatrix} \begin{bmatrix} -\frac{1}{13} & \frac{3}{13} \\ \frac{5}{13} & -\frac{2}{13} \end{bmatrix}$

2.1.1 Matrix Inversion Method

Consider a set of three simultaneous linear algebraic equations

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_3 \end{aligned} \tag{2.7}$$

Equation (2.7) can be expressed in the matrix form

$$\mathbf{Ax} = \mathbf{b}$$

Premultiplying by the inverse $\mathbf{A}^{-1}$, we obtain the solution of x as

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} \tag{2.9}$$

If the matrix $\mathbf{A}$ is non-singular, that is, if $\det (\mathbf{A})$ is not equal to zero, then Eq. (2.9) has a unique solution.

The solution for x_1 is

$$x_1 = \frac{1}{|\mathbf{A}|} \begin{vmatrix} b_1 & a_{12} & a_{13} \\ b_2 & a_{22} & a_{23} \\ b_3 & a_{32} & a_{33} \end{vmatrix} = \frac{1}{|\mathbf{A}|} \left\{ b_1 \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - b_2 \begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} + b_3 \begin{vmatrix} a_{12} & a_{13} \\ a_{22} & a_{23} \end{vmatrix} \right\}$$

$$= \frac{1}{|\mathbf{A}|} \{ b_1 C_{11} + b_2 C_{21} + b_3 C_{31} \}$$

where $\mathbf{A}$ is the determinant of the coefficient matrix $\mathbf{A}$, and $\mathbf{C}_{11}$, $\mathbf{C}_{21}$ and $\mathbf{C}_{31}$ are the cofactors of $\mathbf{A}$ corresponding to element 11, 21 and 31. We can also write similar expressions for x_2 and x_3 by replacing the second and third columns by the y column respectively. Hence, the complete solution can be written in matrix form as follows

$$\begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix} = \frac{1}{|\mathbf{A}|} \begin{bmatrix} C_{11} & C_{21} & C_{31} \\ C_{12} & C_{22} & C_{32} \\ C_{13} & C_{23} & C_{33} \end{bmatrix} \begin{Bmatrix} b_1 \\ b_2 \\ b_3 \end{Bmatrix} \quad (2.10)$$

$$\text{or} \quad \{x\} = \frac{1}{|\mathbf{A}|} [C_{ji}] = \frac{1}{|\mathbf{A}|} [\text{adj} \mathbf{A}] \{b\}$$

$$\text{Hence} \quad \mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \text{adj} \mathbf{A} \text{ and } \text{Adj} \mathbf{A} = \mathbf{A}^{-1} \text{abs}[\mathbf{A}]$$

Although this method is quite general but it is not quite suitable for large systems because evaluation of $\mathbf{A}^{-1}$ by co-factors becomes very cumbersome

Example 1 :

Obtain the solution of the following linear simultaneous equations by the matrix inversion method.

$$\begin{bmatrix} 1 & 3 \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 12 \end{bmatrix}$$

Solution:

$$\begin{bmatrix} 1 & 3 \\ 4 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 12 \end{bmatrix}$$

$$C_{11} = (-1)^{1+1}|-1| = -1$$

$$C_{12} = (-1)^{1+2}|4| = -4$$

$$C_{21} = (-1)^{1+2}|3| = -3$$

$$C_{22} = (-1)^{2+2}|1| = 1$$

$$\text{Hence } \mathbf{C} = \begin{bmatrix} -1 & -4 \\ -3 & 1 \end{bmatrix}$$

$$C^T = \begin{bmatrix} -1 & -3 \\ -4 & 1 \end{bmatrix}$$

$$\mathbf{A}^{-1} = \frac{\mathbf{C}}{|\mathbf{A}|} = -\frac{1}{13} \begin{bmatrix} -1 & -3 \\ -4 & 1 \end{bmatrix}$$

$$\begin{aligned} \text{Hence } \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} &= -\frac{1}{13} \begin{bmatrix} -1 & -3 \\ -4 & 1 \end{bmatrix} \begin{bmatrix} 5 \\ 12 \end{bmatrix} \\ &= -\frac{1}{13} \begin{bmatrix} -5 & -36 \\ -20 & 12 \end{bmatrix} = -\frac{1}{13} \begin{bmatrix} -41 \\ -8 \end{bmatrix} \end{aligned}$$

$$\text{Therefore, } x_1 = \frac{-41}{-13} = \mathbf{3.15}$$

$$\text{and } x_2 = \frac{-8}{-13} = \mathbf{0.62}$$

Example 2:

Find the inverse of the matrix:

$$\mathbf{A} = \begin{bmatrix} 2 & 2 & 0 \\ -2 & 1 & 1 \\ 3 & 0 & 1 \end{bmatrix}$$

and solve the system of equations $[\mathbf{A}]\{x\} = \{b\} = \begin{Bmatrix} 6 \\ 3 \\ 6 \end{Bmatrix}$

Solution:

$$\text{The determinant of } \mathbf{A} = \begin{vmatrix} 2 & 2 & 0 \\ 2 & 1 & 1 \\ 3 & 0 & 1 \end{vmatrix}$$

$$= 2[1(1) - 0(1)] + 2[2(1) - 0(0)] + 3[2(1) - 1(0)] = 12$$

Since $\det A = 12 \neq 0$, the given matrix is non-singular. The cofactors corresponding to the entries in each row of $\det A$ are:

$$C_{11} = \begin{vmatrix} 1 & 1 \\ 0 & 1 \end{vmatrix} = 1 \quad C_{12} = \begin{vmatrix} -2 & 1 \\ 3 & 1 \end{vmatrix} = 5 \quad C_{13} = \begin{vmatrix} -2 & 1 \\ 3 & 0 \end{vmatrix} = -3$$

$$C_{21} = - \begin{vmatrix} -2 & 0 \\ 0 & 1 \end{vmatrix} = -2 \quad C_{22} = \begin{vmatrix} -2 & 0 \\ 3 & 1 \end{vmatrix} = 2 \quad C_{23} = - \begin{vmatrix} 2 & 2 \\ 3 & 0 \end{vmatrix} = 6$$

$$C_{31} = \begin{vmatrix} 2 & 0 \\ 1 & 1 \end{vmatrix} = 2 \quad C_{32} = - \begin{vmatrix} 2 & 0 \\ -2 & 1 \end{vmatrix} = -2 \quad C_{33} = \begin{vmatrix} 2 & 2 \\ -2 & 1 \end{vmatrix} = 6$$

$$A^{-1} = \frac{C}{|A|} = -\frac{1}{12} \begin{bmatrix} -1 & -3 \\ -4 & 1 \end{bmatrix}$$

$$A^{-1} = \frac{C}{|A|} = -\frac{1}{12} \begin{pmatrix} 1 & -2 & 2 \\ 5 & 2 & -2 \\ -3 & 6 & 6 \end{pmatrix} = \begin{pmatrix} \frac{2}{12} & -\frac{1}{6} & \frac{1}{6} \\ \frac{5}{12} & \frac{1}{6} & -\frac{1}{6} \\ -\frac{5}{4} & \frac{1}{2} & -\frac{1}{2} \end{pmatrix}$$

It is easy to verify that $A^{-1}A = AA^{-1} = I$

$$\text{Therefore, } \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [A^{-1}]b = \begin{bmatrix} \frac{2}{12} & -\frac{1}{6} & \frac{1}{6} \\ \frac{5}{12} & \frac{1}{6} & -\frac{1}{6} \\ -\frac{5}{4} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix} \begin{bmatrix} 6 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} \frac{2}{12} & -\frac{1}{6} & \frac{1}{6} \\ \frac{5}{12} & \frac{1}{6} & -\frac{1}{6} \\ -\frac{5}{4} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

$$\begin{bmatrix} 6 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} \frac{2}{12} & -\frac{1}{6} & \frac{1}{6} \\ \frac{5}{12} & \frac{1}{6} & -\frac{1}{6} \\ -\frac{5}{4} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix} \begin{bmatrix} 6 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} \frac{6}{12} & -\frac{3}{6} & \frac{6}{6} \\ \frac{30}{12} & \frac{3}{6} & -\frac{6}{6} \\ -\frac{6}{4} & \frac{3}{2} & -\frac{6}{2} \end{bmatrix} = \begin{bmatrix} \frac{6-6+12}{12} \\ \frac{30+6-12}{12} \\ \frac{-6+6+12}{4} \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

AUGMENTED MATRIX

A system of linear equations in matrix notation takes the form $Ax = b$, where A is of order mn , x is of order $n \times 1$. The augmented matrix $[Ab]$ can be obtained by adjoining column b to matrix A . In terms of partitioned matrices we have $[A_b] = [A : b]$.

As an example let us consider a set of linear equations

$$x + y + z = 8$$

$$x + y - z = 5$$

$$x - y + z = 2$$

Then we have, $[A] = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix}$ $\{b\} = \begin{Bmatrix} 8 \\ 5 \\ 2 \end{Bmatrix}$

and $[A_b] = \begin{bmatrix} 1 & 1 & 1 & 8 \\ 1 & 1 & -1 & 5 \\ 1 & -1 & 1 & 2 \end{bmatrix}$

Convergence Of Matrix Inversion Method

Matrix inversion is another direct method for solving systems of linear equations. It involves finding the inverse of the coefficient matrix and then multiplying it by the vector of constants to obtain the solution vector.

The convergence of matrix inversion method is not discussed in the same way as iterative methods because it's a direct method. Instead, the main concerns with matrix inversion are computational complexity and numerical stability.

Computational Complexity: Inverting a matrix has a computational complexity of approximately $O(n^3)$ where n is the size of the matrix. This makes it computationally expensive for large matrices compared to other methods.

Numerical Stability: Numerical stability can be a concern, especially for matrices that are ill-conditioned or nearly singular. In such cases, small numerical errors can be amplified during the inversion process, leading to inaccurate results.

Due to its computational complexity and sensitivity to numerical errors, matrix inversion is often not preferred for solving systems of linear equations, especially for large or ill-conditioned systems. Instead, other methods like Gaussian elimination, LU decomposition, Cholesky decomposition, or iterative methods such as the Gauss-Seidel method are often preferred.

2.1.2 Gauss Elimination Method

Consider the following system of linear simultaneous equations:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \quad (2.12)$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2 \quad (2.13)$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \quad (2.14)$$

Gauss elimination is a popular technique for solving simultaneous linear algebraic equations. It reduces the coefficient matrix into an upper triangular matrix through a sequence of operations carried out on the matrix. The vector b is also modified in the process. The solution vector $\{x\}$ is obtained from a backward substitution procedure

Two linear systems $Ax = b$ and $A'x = b'$ of equations are said to be equivalent if any solution of one is a solution of the other. Also, let $Ax = b$ is a linear non-homogeneous system of n equations. Suppose we subject this system to the system of following operations:

1. Multiplication of one equation by a non-zero constant.
2. Addition of a multiple of one equation to another equation.
3. Interchange of two equations.

If the sequence of operations produce the new system $A'x=b'$, then both the systems $Ax=b$ and $A'x = b'$ are equivalent. In particular, then A is invertible if A' is invertible. In Gauss elimination method, we adopt this and the elimination process is based on this theorem.

In Gauss elimination method, the unknowns are eliminated such that the elimination process leads to an upper triangular system and the unknowns are obtained by back substitution. It is assumed $a_{11} \neq 0$. This method can be described by the following steps:

Step1: Eliminate x_1 from the second and third equations. Using the first equation (2.12), the following operations are performed:

$$(2.13) - \left(\frac{a_{21}}{a_{11}}\right) (2.12) \text{ and } (2.14) - \left(\frac{a_{31}}{a_{11}}\right) - (2.12)$$

$$\text{gives} \quad a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \quad (2.15)$$

$$a'_{22}x_2 + a'_{23}x_3 = b'_2 \quad (2.16)$$

$$a'_{32}x_2 + a'_{33}x_3 = b'_3 \quad (2.17)$$

Equation (2.15) is called the pivotal equation and the coefficient a_{11} is the pivot.

Step 2: Eliminate x_2 from the Eq. (2.17) using Eq. (2.16) by assuming a $c \neq 0$. We perform the following operation:

$$(2.17) - \left(\frac{a'_{32}}{a'_{22}} \right) (2.16)$$

$$\text{to obtain} \quad a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \quad (2.15)$$

$$a'_{22}x_2 + a'_{23}x_3 = b'_2 \quad (2.16)$$

$$\text{and} \quad a'_{33}x_3 = b''_3 \quad (2.17)$$

Here Eq. (2.19) is called the *pivotal* equation and the coefficient a'_{22} is the pivot.

Step 3: To find x_1, x_2 and x_3 , we apply back substitution starting from Eq. (2.20) giving x_3 , then x_2 , from Eq. (2.19) and x_1 from Eq. (2.18).

Pivoting : Gauss elimination method fails if any one of the pivots in the above equations (2.12) to (2.20) becomes zero. To overcome this difficulty, the equations are to be rewritten in a slightly different order such that the pivots are not zero.

Partial pivoting method : Step 1: The numerically largest coefficient of x_1 is selected from all the equations and the corresponding equation becomes the first equation (2.12).

Step 2: The numerically largest coefficient of x_2 is selected from all the remaining equations as pivot and the corresponding equation becomes the second equation (2.16). This process is repeated till an equation into a simple variable is obtained.

Complete pivoting method: In this method, we select at each stage the numerically largest coefficient of the complete matrix of coefficients. This procedure leads to an interchange of the equations as well as interchange of the position of variables.

Example:

Solve The Following Equation By Gauss-Elimination Method

$$2x + 4y - 6z = -4$$

$$x + 5y + 3z = 10$$

$$x + 3y + 2z = 5$$

Solution:

$$2x + 4y - 6z = -4 \quad (\text{E.1})$$

$$x + 5y + 3z = 10 \quad (\text{E.2})$$

$$x + 3y + 2z = 5 \quad (\text{E.3})$$

To eliminate x from (E.2) and (E.3) using (E.1):

$$2x + 4y - 6z = -4$$

$$x + 5y + 3z = 10 \quad 1 \times (-2)$$

$$x + 3y + 2z = 5 \quad 1 \times (-2)$$

$$2x + 4y - 6z = -4$$

$$-2x - 10y - 6z = -20$$

$$-2x - 6y - 4z = -10$$

$$2x + 4y - 6z = -4$$

$$\text{Row 1} + \text{Row 2: } -6y - 12z = -24 \quad (\text{E.6})$$

$$\text{Row 1} + \text{Row 3: } 2y - 10z = -14 \quad 1 \times (-3)$$

To eliminate y from (E.5) using (E.4):

$$2x + 4y - 6z = -4$$

$$-6y - 12z = -24$$

$$6y + 30z = 42$$

$$2x + 4y - 6z = -4$$

$$-6y - 12z = -24$$

$$\text{Row 2} + \text{Row 3: } 18z = 18 \implies z=1$$

valuation of the unknowns by back substitution:

$$-6y - 12z = -24 \quad 6y = 24 - 12z$$

$$\implies y = \frac{24-12 \times 1}{6} \quad \implies y = 2$$

$$2x + 4y - 6z = -4 \quad 2x = -4 - 4y + 6z$$

$$\implies x = \frac{-4-4 \times 2+6 \times 1}{2} \quad \implies x = -3$$

Example:

Using the Gaussian elimination method, solve the system of equations

$$[A] \mathbf{x} = \mathbf{b} \text{ where } [A] = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & -1 & 3 & 0 \\ 0 & 2 & 0 & 3 \\ -1 & 0 & 2 & 1 \end{bmatrix} \quad \{\mathbf{b}\} = \begin{Bmatrix} 3 \\ 3 \\ 1 \\ 0 \end{Bmatrix}$$

Solution:

The augmented matrix is

$$[A_b] = \begin{bmatrix} 1 & 1 & 1 & 1 & 3 \\ 2 & -1 & 3 & 0 & 3 \\ 0 & 2 & 0 & 3 & 1 \\ -1 & 0 & 2 & 1 & 0 \end{bmatrix}$$

From the augmented matrix, we apply elementary transformations:

Row 2 -2 × Row 1

Row 4 + Row 1

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 3 \\ 0 & -3 & 1 & -2 & 3 \\ 0 & 2 & 0 & 3 & 1 \\ 0 & 2 & 0 & 3 & 1 \\ 0 & 1 & 3 & 2 & 3 \end{bmatrix}$$

which gives

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 3 \\ 0 & -3 & 1 & -2 & 3 \\ 0 & 2 & 0 & 3 & 1 \\ 0 & 0 & \frac{2}{3} & \frac{5}{3} & -1 \\ 0 & 0 & \frac{10}{3} & \frac{4}{3} & 3 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 1 & 1 & 3 \\ 0 & -3 & 1 & -2 & 3 \\ 0 & 2 & 0 & 3 & 1 \\ 0 & 0 & \frac{2}{3} & \frac{5}{3} & -1 \\ 0 & 0 & 0 & -7 & 7 \end{bmatrix}$$

Hence, by back substitution the upper triangular matrix, we obtain

$$x_4 = -1, x_3 = 1, x_2 = 2, x_1 = 1$$

Convergence Of Gauss Elimination Method

Gaussian elimination is a direct method used to solve systems of linear equations by transforming the augmented matrix of the system into row-echelon form and then back-substituting to find the solution. Since it is a direct method, convergence is not a concern.

Here's a basic outline of Gaussian elimination:

Start with the augmented matrix, which combines the coefficient matrix and the constant terms. Use elementary row operations (such as multiplying a row by a constant, adding one row to another, or swapping rows) to transform the matrix into upper triangular form. Once in upper triangular form, use back-substitution to find the values of the variables. Convergence is not an issue with Gaussian elimination because it is a deterministic algorithm that follows a fixed set of steps.

However, as with other numerical methods, numerical stability can be a concern. This is especially true when dealing with ill-conditioned matrices or matrices with very small pivots. In such cases, round-off errors or other numerical issues may affect the accuracy of the computed solution.

Overall, Gaussian elimination is a reliable method for solving systems of linear equations, but attention should be paid to numerical stability when implementing it in practice, especially for large or ill-conditioned systems.

2.1.3 Gauss-Jordan Method

Gauss-Jordan method is an extension of the Gauss elimination method. The set of equations $Ax = b$ is reduced to a diagonal set $Ix = b'$, where I is a unit matrix. This is equivalent to $x = b'$. The solution vector is therefore obtained directly from b' . The Gauss-Jordan method implements the same series of operations as implemented by Gauss elimination process. The main difference is that it applies these operations below as well as above the diagonal such that all off-diagonal elements of the matrix are reduced to zero. Gauss-Jordan method also provides the inverse of the coefficient matrix A along with the solution vector $\{x\}$. The Gauss-Jordan method is highly used due to its stability and direct procedure. The Gauss-Jordan method requires more computational effort than Gauss elimination process.

Gauss-Jordan method is a modification of Gauss elimination method. The series of operations performed are quite similar to the Gauss elimination method. In the Gauss elimination method, an upper triangular matrix is derived while in the Gauss-Jordan method an identity matrix is derived. Hence, back substitutions are not required.

Example:

Solve the following equations by Gauss-Jordan method.

$$\begin{aligned}x + 3y + 2z &= 17 \\x + 2y + 3z &= 16 \\2x - y + 4z &= 1\end{aligned}$$

Solution:

$$\begin{aligned}\text{consider,} \\x + 3y + 2z &= 17\end{aligned}\tag{E.1}$$

$$x+2y+3z=16 \quad (\text{E.2})$$

$$2x-y+4z=13 \quad (\text{E.3})$$

$$x + 3y + 2z = 17 \quad (\text{E.1}) \quad (-2) + (\text{E.3})$$

$$x + 2y + 3z = 16 \quad (\text{E.2}) \quad (-1) + (\text{E.1})$$

$$2x - y + 4z = 13$$

$$x + 3y + 2z = 17$$

$$y - z = 1 \quad x(2) + (\text{E.1})$$

$$7y = 21 \implies y = 3$$

$$x + 5y = 19$$

$$y - z = 1 \implies yz = y-1 = 3-1 \implies yz = 2$$

$$x + 5y = 19 \implies x = 19 - 5 \times 3 \implies x = 4$$

Example:

Solve the following equations by Gauss-Jordan method.

$$x - 2y = -4$$

$$5y + z = -9$$

$$4x - 3z = -10$$

Solution:

$$\text{The augmented matrix is } \left[\begin{array}{ccc|c} 1 & -2 & 0 & -4 \\ 0 & -5 & 1 & -9 \\ 4 & 0 & -3 & -10 \end{array} \right]$$

Multiplying 1_{st} row by -4 and adding the result to the 3_{rd} row, we obtain

$$-4R_1 + R_3 \longrightarrow \left[\begin{array}{ccc|c} 1 & -2 & 0 & -4 \\ 0 & -5 & 1 & -9 \\ 0 & 8 & -3 & 6 \end{array} \right]$$

Now, multiply the 2^{nd} row by $-\frac{1}{5}$

$$-\frac{1}{5}R_2 \longrightarrow \left[\begin{array}{ccc|c} 1 & -2 & 0 & -4 \\ 0 & 1 & -\frac{1}{5} & \frac{9}{5} \\ 0 & 8 & -3 & 6 \end{array} \right]$$

Multiply the 2_{nd} row and add the result to the 1_{st} row. Then multiply the 2_{nd} row by -8 and add the result to the 3_{rd} row.

$$2R_2 + R_1 \longrightarrow \left[\begin{array}{ccc|c} 1 & -2 & -\frac{2}{5} & -\frac{2}{5} \\ 0 & 1 & -\frac{1}{5} & \frac{9}{5} \\ 0 & 0 & -\frac{7}{5} & -\frac{42}{5} \end{array} \right]$$

Multiply 3_{rd} row by $-5/7$

$$-5/7R_3 \longrightarrow \left[\begin{array}{ccc|c} 1 & -2 & -\frac{2}{5} & -\frac{2}{5} \\ 0 & 1 & -\frac{1}{5} & \frac{9}{5} \\ 0 & 0 & 1 & 6 \end{array} \right]$$

Multiply 3_{rd} row by $2/5$ and add the result to 1_{st} row. Then multiply 3_{rd} row by $1/5$ and add the result to 2_{nd} row.

$$\begin{aligned} \frac{2}{5}R_3 + R_1 &\longrightarrow \\ \frac{1}{5}R_3 + R_2 &\longrightarrow \left[\begin{array}{ccc|c} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 0 & 6 \end{array} \right] \end{aligned}$$

Hence, the last matrix above represents the system with $x = 2$, $y = 3$ and $z = 6$.

LU Decomposition:

It is possible to show that any square matrix A can be expressed as a product of a lower triangular matrix L and an upper triangular matrix U .

$$A = LU$$

For instance

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix}$$

The process of computing L and U for a given A is known as LU Decomposition or LU Factorisation. LU decomposition is not unique

(the combinations of L and U for a prescribed A are endless), unless certain constraints are placed on L or U . These constraints distinguish one type of decomposition from another. Two commonly used decompositions are given below:

1. Cholesky's decomposition: Constraints are $L = U^T$
2. Crout's decomposition: Constrains are $U_{ii} = 1, i = 1, 2, \dots, n$. After decomposing the matrix A , it is easier to solve the equations $Ax = b$. We can rewrite the equations as

$$LU_x = b$$

or denoting $U_x = y$, the above equation becomes

$$L_y = b$$

This equation $L_y = b$ can be solved for y by forward substitution. Then $U_x = y$ will yield x by the backward substitution process. The advantage of LU decomposition method over the Gauss elimination method is that once A is decomposed, we can solve $Ax = b$ for as many constant vectors b as we please. Also, the forward and backward substitutions operations are much less time consuming than the decomposition process.

Convergence Of Gauss-Jordan Method

The Gauss-Jordan method, also known as row reduction or the method of elimination, is a direct method for solving systems of linear equations by transforming the augmented matrix representing the system into reduced row-echelon form. It's an exact method, not an iterative one, so convergence is not a concern.

Here's a brief overview of the process:

Start with the augmented matrix, which combines the coefficient matrix and the constant terms. Perform row operations to transform the matrix into reduced row-echelon form, where each leading entry of a row is 1, each leading entry is the only non-zero entry in its column, and all entries above and below leading entries are zero. Once in reduced row-echelon form, the solution to the system of equations is directly read from the augmented matrix. Since the Gauss-Jordan method is a deterministic algorithm that follows a fixed set of steps, convergence is not an issue. It's guaranteed to produce the exact solution to the system of linear equations if one exists. However, like other methods, numerical stability can be a concern, especially when

dealing with ill-conditioned matrices or matrices with very small pivots. In practice, care should be taken to handle such situations to ensure accurate results.

2.1.4 Cholesky's Triangularisation Method

Cholesky's decomposition method is faster than the LU decomposition. There is no need for pivoting. If the decomposition fails, the matrix is not positive definite.

Consider the system of linear equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned} \tag{2.21}$$

$$\text{The above system can be written as} \tag{2.22}$$

$$\mathbf{Ax}=\mathbf{b}$$

$$\text{where } \mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}, \mathbf{X} = \begin{Bmatrix} x_1 \\ x_2 \\ x_3 \end{Bmatrix}, \mathbf{b} = \begin{Bmatrix} b_1 \\ b_2 \\ b_3 \end{Bmatrix}$$

$$\text{Let } \mathbf{A} = \mathbf{LU} \dots \tag{2.23}$$

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \text{ and } \mathbf{U} = \begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix}$$

Equation (2.21) can be written as

$$\mathbf{LUX} = \mathbf{b} \tag{2.24}$$

$$\text{If we write } \mathbf{UX} = \mathbf{V} \tag{2.25}$$

Equation (2.24) becomes

$$\mathbf{LV} = \mathbf{b} \tag{2.26}$$

Equation (2.26) is equivalent to the system

$$v_1 = b_1$$

$$l_{21}v_1 + v_2 = b_2$$

$$l_{31}v_1 + l_{32}v_2 + v_3 = b_3 \tag{2.27}$$

The above system can be solved to find the values of v_1 , v_2 and v_3 which

give us the matrix V .

$$UX = V$$

then becomes,

$$u_{11}x_1 + u_{12}x_2 + u_{13}x_3 = v_1$$

$$u_{22}x_2 + u_{23}x_3 = v_2$$

$$u_{33}x_3 = v_3 \quad (2.28)$$

which can be solved for x_3 , x_2 and x_1 by the backward substitution process.

In order to compute the matrices L and U , we write Eq. (2.23) as

$$\begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad (2.29)$$

Multiplying the matrices on the left and equating the corresponding elements of both sides, we obtain

$$u_{11} = a_{11}, u_{12} = a_{12}, u_{13} = a_{13} \quad (2.30)$$

$$l_{21}u_{11} = a_{21} \Rightarrow l_{21} = \frac{a_{21}}{a_{11}}$$

$$l_{31}u_{11} = a_{31} \Rightarrow l_{31} = \frac{a_{31}}{a_{11}}$$

$$l_{21}u_{12} + u_{22} = a_{22} \Rightarrow u_{22} = a_{22} - \frac{a_{21}}{a_{11}} a_{12}$$

$$l_{21}u_{13} + u_{23} = a_{23} \Rightarrow u_{23} = a_{23} - \frac{a_{21}}{a_{11}} a_{13}$$

$$l_{31}u_{12} + l_{32}u_{22} = a_{32} \Rightarrow l_{32} = \frac{1}{u_{22}} \left[a_{32} - \frac{a_{31}}{a_{11}} a_{12} \right] \quad (2.33)$$

$$\text{and} \quad l_{31}u_{13} + l_{32}u_{23} + u_{33} = a_{33} \quad (2.34)$$

The value of u_{33} can be computed from Eq. (2.34). To obtain the elements of L and U , we first find the first row of U and the first column of L . Then, we determine the second row of U and the second column of L . Finally, we compute the third row of U .

Cholesky's triangularisation method is also known as Crout's triangularisation method or method of factorisation.

Example:

Solve the following equations by Cholesky's triangularisation method

$$2x + y + 4z = 12$$

$$8x - 3y + 2z = 20$$

$$4x + 11y - z = 33$$

Solution:

We have
$$A = \begin{bmatrix} 2 & 1 & 4 \\ 8 & -3 & 2 \\ 4 & 11 & -1 \end{bmatrix}, X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, B = \begin{bmatrix} 12 \\ 20 \\ 33 \end{bmatrix}$$

Let
$$\begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix} = \begin{bmatrix} 2 & 1 & 4 \\ 8 & -3 & 2 \\ 4 & 11 & -1 \end{bmatrix}$$

Multiplying and equating we get:

$$l \times u_{11} = 2 \Rightarrow u_{11} = 2$$

$$l \times u_{12} = 1 \Rightarrow u_{12} = 1$$

$$l \times u_{13} = 4 \Rightarrow u_{13} = 4$$

$$l_{21} \times u_{11} = 8 \Rightarrow l_{21} = \frac{8}{u_{11}} = \frac{8}{2} = 4$$

$$l_{21} \times u_{12} + u_{22} = -3 \Rightarrow u_{22} = -3 - l_{21} \times u_{12} = -3 - 4 \times 1 = -7$$

$$l_{21} \times u_{13} + u_{23} = 2 \Rightarrow u_{23} = 2 - l_{21} \times u_{13} = 2 - 4 \times 4 = -14$$

$$l_{31} \times u_{11} = 4 \Rightarrow l_{31} = \frac{4}{u_{11}} = \frac{4}{2} = 2$$

$$l_{31} \times u_{12} + l_{32} \times u_{22} = 11 \Rightarrow l_{32} = \frac{11 - l_{31} \times u_{12}}{u_{22}} = \frac{11 - 2 \times 1}{-7} = -\frac{9}{7}$$

$$l_{31} \times u_{13} + l_{32} \times u_{23} + l_{33} \times u_{33} = -1 \Rightarrow u_{33} = -1 - l_{31} \times u_{13} - l_{32} \times u_{23}$$

$$= -1 - 2 \times 4 - \left(-\frac{9}{7}(-14)\right) = -27$$

We get:
$$A = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 2 & -\frac{9}{7} & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 4 \\ 0 & -7 & -14 \\ 0 & 0 & -27 \end{bmatrix}$$

and the given system can be written as:

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 2 & -\frac{9}{7} & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 4 \\ 0 & -7 & -14 \\ 0 & 0 & -27 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 12 \\ 20 \\ 30 \end{bmatrix}$$

Writing: $LV = B$, we get

$$\begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 2 & -\frac{9}{7} & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 4 \\ 0 & -7 & -14 \\ 0 & 0 & -27 \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ V_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 20 \\ 30 \end{bmatrix}$$

which gives $V_1 = 12 \Rightarrow V_2$

$$4V_1 + V_2 = 20 \Rightarrow V_2 = 20 - 4 \times 12 = -28$$

$$2V_1 - \frac{9}{7}V_2 + V_3 \Rightarrow V_3 = 33 + \frac{9}{7}(-28) - 2 \times 12 = -27$$

The solution to the original system is given by:

$$UX = V$$

$$\begin{bmatrix} 2 & 1 & 4 \\ 0 & -7 & -14 \\ 0 & 0 & -27 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 12 \\ -28 \\ -27 \end{bmatrix}$$

$$2x + y + 4z = 12$$

$$-7y - 14z = -28$$

$$-27z = -27 \Rightarrow z = 1$$

$$7y = 28 - 14 \times 1 \Rightarrow y = \frac{14}{7} \Rightarrow y = 2$$

$$2x = 12 - y - 4z = 12 - 2 - 4 \times 1 \Rightarrow x = \frac{6}{2} \Rightarrow x = 3$$

Convergence Of Cholesky's Method

Cholesky decomposition is another direct method for solving systems of linear equations, particularly applicable to symmetric positive definite matrices. Unlike iterative methods such as Jacobi or Gauss-Seidel, Cholesky decomposition directly computes $A=LL^T$, where L is a lower triangular matrix. This decomposition allows for

efficient solution of linear systems by forward and backward substitution.

Convergence is not a concern for Cholesky decomposition, as it is a direct method and does not involve iteration. However, the applicability and correctness of Cholesky decomposition depend on the properties of the matrix being decomposed:

Symmetry: The matrix must be symmetric for Cholesky decomposition to be applicable.

Positive Definiteness: The matrix must be positive definite. This condition ensures that the decomposition yields a valid lower triangular matrix L . A symmetric matrix A is positive definite if $x^T A x > 0$ for all nonzero vectors x .

Numerical Stability: Numerical stability can be a concern due to the potential for round-off errors, especially when dealing with ill-conditioned matrices or matrices with very small pivots. However, Cholesky decomposition is generally considered more stable than methods like LU decomposition for symmetric positive definite matrices.

2.1.5 Crout's Method

This method is based on the fact that every square matrix A can be expressed as the product of a lower triangular matrix and an upper triangular matrix, provided all the principle minors of A are non-singular. Also, such a factorisation, if exists, is unique. This method is also called *triangularisation or factorisation method*. Here, we factorise the given matrix as $A = LU$, where L is a lower triangular matrix with unit diagonal elements and U is an upper triangular matrix. Then,

$$A^{-1} = (LU)^{-1} = U^{-1}L^{-1}$$

Consider the system

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned} \tag{2.35}$$

The above system can be written as

$$Ax=b$$

$$\text{Let} \quad A=LU \tag{2.36}$$

$$\text{Where } \mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \text{ and } \mathbf{U} = \begin{bmatrix} 1 & U_{12} & U_{13} \\ 0 & 1 & U_{23} \\ 0 & 0 & 1 \end{bmatrix} \quad (2.37)$$

Here, $\mathbf{L}$ is a lower triangular matrix and $\mathbf{U}$ is an upper triangular matrix with diagonal elements equal to unity.

$$\mathbf{A} = \mathbf{LU} \Rightarrow \mathbf{A}^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1} \quad (2.38)$$

Now $\mathbf{A} = \mathbf{LU}$

$$\Rightarrow \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} 1 & U_{12} & U_{13} \\ 0 & 1 & U_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

Or

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & l_{11}u_{12} & l_{11}u_{13} \\ l_{21} & l_{21}u_{12} + l_{22} & l_{21}u_{13} + l_{22}u_{23} \\ l_{31} & l_{31}u_{12} + l_{32} & l_{31}u_{13} + l_{32}u_{23} + l_{33} \end{bmatrix}$$

Equating the corresponding elements, we obtain

$$l_{11} = a_{11} \quad l_{21} = a_{21} \quad l_{31} = a_{31} \quad (2.39)$$

$$l_{11}u_{12} = a_{12} \quad l_{11}u_{13} = a_{13} \quad (2.40)$$

$$l_{21}u_{12} + l_{22} = a_{22} \quad l_{31}u_{12} + l_{32} = a_{32} \quad (2.41)$$

$$l_{21}u_{13} + l_{22}u_{23} = a_{23} \quad (2.42)$$

$$\text{and } l_{11}u_{13} + l_{32}u_{23} + l_{33} = a_{33} \quad (2.43)$$

from (2.40) we find

$$u_{12} = a_{12}/l_{11} = a_{12}/a_{11}$$

from (2.41) we obtain

$$l_{22} = a_{22} - l_{21}u_{12} \quad (2.44)$$

$$l_{32} = a_{32} - l_{31}u_{12} \quad (2.45)$$

Equation (2.42) gives

$$u_{23} = (a_{23} - l_{21}u_{13})/l_{22} \quad (2.46)$$

from the relation (2.43) we get

$$l_{33} = a_{33} - l_{31}u_{13} - l_{32}u_{23} \quad (2.47)$$

Thus, we have determined all the elements of L and U.

From Eqs.(2.36) and (2.37) we have

$$LU_x = b \quad (2.48)$$

Let $UX = V$

$$\text{Where } V = \begin{bmatrix} V_1 \\ V_2 \\ \vdots \\ V_n \end{bmatrix}$$

From Equation (2.48) we have $LV = b$, which on forward substitution yields V.

From $UX = V$, we find x (by backward substitution)

Example:

Solve the following set of equations by Crout's method.

$$2x + y + 4z = 12$$

$$8x - 3y + 2z = 20$$

$$4x + 11y - z = 33$$

Solution:

$$\text{We have, } A = \begin{bmatrix} 2 & 1 & 4 \\ 8 & -3 & 2 \\ 4 & 11 & -1 \end{bmatrix}, X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, B = \begin{bmatrix} 12 \\ 20 \\ 33 \end{bmatrix}$$

$$AX = B$$

Let $A = L$

$$L = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \text{ and } U = \begin{bmatrix} 1 & U_{12} & U_{13} \\ 0 & 1 & U_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & 4 \\ 8 & -3 & 2 \\ 4 & 11 & -1 \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} 1 & U_{12} & U_{13} \\ 0 & 1 & U_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & 4 \\ 8 & -3 & 2 \\ 4 & 11 & -1 \end{bmatrix} = \begin{bmatrix} l_{11} & l_{11}u_{12} & l_{11}u_{13} \\ l_{21} & l_{21}u_{12} + l_{22} & l_{21}u_{13} + l_{22}u_{23} \\ l_{31} & l_{31}u_{12} + l_{32} & l_{31}u_{13} + l_{32}u_{23} + l_{33} \end{bmatrix}$$

$$l_{11}u_{12} = \mathbf{1} \Rightarrow u_{12} = \frac{1}{2}$$

$$l_{11}u_{13} = \mathbf{4} \Rightarrow u_{13} = \frac{4}{2} = 2$$

$$l_{22} + l_{21}u_{12} = -\mathbf{3} \Rightarrow l_{22} = -\mathbf{3} - \mathbf{8} \left(\frac{1}{2}\right) = -\mathbf{7}$$

$$l_{32} + l_{31}u_{12} = -\mathbf{3} \Rightarrow l_{32} = \mathbf{11} - \mathbf{4} \left(\frac{1}{2}\right) = \mathbf{9}$$

$$l_{21}u_{13} + l_{22}u_{23} = \mathbf{2} \Rightarrow u_{23} = \frac{2-8 \times 2}{-7} = \mathbf{2}$$

$$l_{31}u_{13} + l_{32}u_{23} + l_{33} = -\mathbf{1} \Rightarrow l_{33} = -1 - 4 \times 2 - 9 \times 2 = -\mathbf{27}$$

$$\mathbf{L} = \begin{bmatrix} 2 & 0 & 0 \\ 8 & -7 & 0 \\ 4 & 9 & -27 \end{bmatrix} \text{ and } \mathbf{U} = \begin{bmatrix} 1 & \frac{1}{2} & 2 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{LV} = \mathbf{B}$$

$$\mathbf{L} = \begin{bmatrix} 2 & 0 & 0 \\ 8 & -7 & 0 \\ 4 & 9 & -27 \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \\ V_3 \end{bmatrix} = \begin{bmatrix} 12 \\ 20 \\ 33 \end{bmatrix}$$

$$2V_1 = \mathbf{12} \Rightarrow V_1 = \mathbf{6}$$

$$8V_1 - 7V_2 = \mathbf{20} \Rightarrow V_2 = \frac{-20+8 \times 6}{7} = \mathbf{4}$$

$$4V_1 + 9V_2 - 27V_3 = \mathbf{33} \Rightarrow V_3 = \frac{-33+4 \times 6+9 \times 4}{27} = \mathbf{1}$$

$$\mathbf{V} = \begin{bmatrix} V_1 \\ V_2 \\ V_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 1 \end{bmatrix} ; \mathbf{Ux} = \mathbf{V}$$

$$\begin{bmatrix} 1 & \frac{1}{2} & 2 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 1 \end{bmatrix}$$

$$x + \frac{1}{2}y + 2 = 6$$

$$y + 2z = 4$$

$$\Rightarrow z = 1$$

$$y = 4 - 2 \times 1$$

$$\Rightarrow y = 2$$

$$x = 6 - \frac{1}{2} \times 2 - 2 \times 1 \Rightarrow x = 3$$

Convergence Of Croute's Method

Crout's method is not an iterative method like the Jacobi or Gauss-Seidel methods; instead, it's a direct method for solving systems of linear equations using LU decomposition.

In Crout's method, a matrix A is decomposed into the product of a lower triangular matrix L and an upper triangular matrix U such that $A=LU$. Once this decomposition is achieved, solving the linear system $Ax=b$ reduces to solving two triangular systems $Ly=b$ for y and $Ux=y$ for x.

The convergence of Crout's method is not a concern as it's not an iterative method. Instead, the focus is on the accuracy and stability of the LU decomposition process. The LU decomposition is guaranteed for nonsingular matrices, meaning those with a nonzero determinant, and it can be computed efficiently using techniques like Gaussian elimination.

The stability and accuracy of Crout's method can be influenced by factors such as round-off errors during the decomposition process, the condition number of the matrix A, and the presence of large or small pivots during Gaussian elimination. These factors can affect the numerical stability and accuracy of the solution obtained using Crout's method.

2.1.6 Jacobi's Iteration Method

This method is also known as the method of simultaneous displacements. Consider the system of linear equations

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$

$$\begin{aligned} a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned} \quad (2.49)$$

Here, we assume that the coefficients a_{11} , a_{22} and a_{33} are the largest coefficients in the respective equations so that

$$\begin{aligned} |a_{11}| &> |a_{12}| + |a_{13}| \\ |a_{21}| &> |a_{22}| + |a_{23}| \\ |a_{31}| &> |a_{32}| + |a_{33}| \end{aligned} \quad (2.50)$$

Jacobi's iteration method is applicable only if the conditions given in Eq.(2.50) are satisfied.

Now, we can write Eq.(2.49)

$$\begin{aligned} x_1 &= \frac{1}{a_{11}}(b_1 - a_{11}x_2 + a_{13}x_3) \\ x_2 &= \frac{1}{a_{21}}(b_1 - a_{21}x_2 + a_{23}x_3) \\ x_3 &= \frac{1}{a_{31}}(b_1 - a_{31}x_2 + a_{33}x_3) \end{aligned} \quad (2.51)$$

Let the initial approximations be x_1^0 , x_2^0 and x_3^0 respectively. The following iterations are then carried out.

Iteration 1: The first improvements are found as

$$\begin{aligned} x_{11} &= \frac{1}{a_{11}}(b_1 - a_{11}x_2^0 + a_{13}x_3) \\ x_{21} &= \frac{1}{a_{21}}(b_1 - a_{21}x_2^0 + a_{23}x_3) \\ x_{31} &= \frac{1}{a_{31}}(b_1 - a_{31}x_2^0 + a_{33}x_3) \end{aligned} \quad (2.52)$$

Iteration 2: The second improvements are obtained as

$$\begin{aligned} x_{11} &= \frac{1}{a_{11}}(b_1 - a_{11}x_2^0 + a_{13}x_3) \\ x_{22} &= \frac{1}{a_{21}}(b_1 - a_{21}x_2^0 + a_{23}x_3) \\ x_{32} &= \frac{1}{a_{31}}(b_1 - a_{31}x_2^0 + a_{33}x_3) \end{aligned} \quad (2.53)$$

The above iteration process is continued until the values of x_1 , x_2 and x_3 are found to a pre-assigned degree of accuracy. That is, the procedure is continued until the relative error between two consecutive vector norm is satisfactorily small. In Jacobi's method, it is a general practice to assume $x_1^0 = x_2^0 = x_3^0 = 0$. The method can be extended to a

system of n linear simultaneous equations in n unknowns.

Example E2.18

Solve the following equations by Jacobi's method.

$$15x + 3y - 2z = 85$$

$$2x + 10y + z = 51$$

$$x - 2y + 8z = 5$$

Solution:

In the above equations:

$$|15| > |3| + |-2|$$

$$|10| > |2| + |1|$$

$$|8| > |1| + |-2|$$

then Jacobi's method is applicable. We rewrite the given equations as follows:

$$x = \frac{1}{a_1}(d_1 - a_1y - c_1z) = \frac{1}{15}(85 - 3y + 2z)$$

$$y = \frac{1}{b_2}(d_2 - a_2x - c_2z) = \frac{1}{10}(51 - 2x - z)$$

$$z = \frac{1}{c_3}(d_3 - a_3x - b_3y) = \frac{1}{8}(5 - x + 2y)$$

Let the initial approximations be:

$$x^0 = y^0 = z^0 = 0$$

Iteration 1:

$$x_1 = \frac{d_1}{a_1} = \frac{85}{15} = \frac{17}{3}$$

$$y_1 = \frac{d_2}{b_2} = \frac{51}{10}$$

$$z_1 = \frac{d_3}{c_3} = \frac{5}{8}$$

Iteration 2:

$$x_2 = \frac{1}{a_1}(d_1 - b_1y_1 - c_1z_1) = \frac{1}{15} \left(85 - 3 \times \frac{51}{10} - (-2) \times \frac{5}{8} \right)$$

$$x_2 = \mathbf{4.73}$$

$$y_2 = \frac{1}{b_2}(d_2 - a_2x_1 - c_2z_1) = \frac{1}{10} \left(51 - 2 \times \frac{17}{3} - 1 \times \frac{5}{8} \right)$$

$$y_2 = \mathbf{3.904}$$

$$z_2 = \frac{1}{c_3}(d_3 - a_3x_1 - b_3y_1) = \frac{1}{8} \left(5 - 1 \times \frac{17}{3} - (-2) \times \frac{51}{10} \right)$$

$$z_2 = \mathbf{1.192}$$

iteration 3:

$$x_3 = \frac{1}{15} (85 - 3 \times 3.904 + 2 \times 1.192) = \mathbf{5.045}$$

$$y_3 = \frac{1}{10} (51 - 2 \times 4.73 - 1 \times 1.192) = \mathbf{3.99}$$

$$z_3 = \frac{1}{8} (5 - 1 \times 4.173 + 2 \times 3.904) = \mathbf{1.003}$$

Iteration 5:

$$x_5 = \frac{1}{15} (85 - 3 \times 3.99 + 2 \times 1.003) = \mathbf{5.002}$$

$$y_5 = \frac{1}{10} (51 - 2 \times 4.994 - 1 \times 1.003) = \mathbf{4.001}$$

$$z_5 = \frac{1}{8} (5 - 1 \times 4.994 + 2 \times 3.99) = \mathbf{0.998}$$

Iteration 6:

$$x_6 = \frac{1}{15} (85 - 3 \times 4.001 + 2 \times 0.998) = \mathbf{5.0}$$

$$y_6 = \frac{1}{10} (51 - 2 \times 5.002 - 1 \times 0.998) = \mathbf{4.0}$$

$$z_6 = \frac{1}{8} (5 - 1 \times 5.002 + 2 \times 4.001) = \mathbf{1.0}$$

Iteration 7:

$$x_7 = \frac{1}{15} (85 - 3 \times 4 + 2 \times 1) = \mathbf{5.0}$$

$$y_7 = \frac{1}{10} (51 - 2 \times 5 - 1 \times 1) = \mathbf{4.0}$$

$$z_7 = \frac{1}{8} (5 - 1 \times 5 + 2 \times 4) = \mathbf{1.0}$$

Convergence Of Jacobi Method

The convergence of the Jacobi method depends on properties of the coefficient matrix of the linear system, similar to the Gauss-Seidel method:

Diagonal Dominance or Symmetry: Just like Gauss-Seidel, for convergence to occur, the matrix should be diagonally dominant or symmetric positive definite.

Irreducibility: The matrix should be irreducible.

Consistency: The system of linear equations should have a unique solution.

Smoothness of Coefficients: Smooth coefficients generally facilitate faster convergence.

Initial Guess: As with Gauss-Seidel, a good initial guess can accelerate convergence.

However, compared to Gauss-Seidel, the Jacobi method has a weaker convergence property. Specifically, the Jacobi method converges if the coefficient matrix is strictly diagonally dominant, meaning that the absolute value of each diagonal element is greater than the sum of the absolute values of the other elements in the corresponding row. This condition is stricter than the general diagonal dominance condition required for Gauss-Seidel.

Despite its weaker convergence property, the Jacobi method is still widely used, particularly when parallel computing is employed, due to its simplicity and ease of parallelization. Like any iterative method, its convergence can be influenced by factors such as the choice of initial guess, the properties of the coefficient matrix, and the number of iterations allowed.

2.1.7 Gauss-Seidal Iteration Method

The Gauss-Seidal method is applicable to predominantly diagonal systems. A predominantly diagonal system has large diagonal elements. The absolute value of the diagonal element in each case is larger than the sum of the absolute values of the other elements in that row of the matrix A . For such predominantly diagonal systems, the Gauss-Seidal method always converges to the correct solution, irrespective of the choice of the initial estimates. Since the most recent approximations of the variables are used while proceeding to the next step, the convergence of the Gauss-Seidal method is twice as fast as in Jacobi's method. The Gauss-Seidal and Jacobi's methods converge for any choice of the initial approximations, if in each equation of the system, the absolute value of the largest coefficient is greater than the sum of

the absolute values of the remaining coefficients. In other words,

$$\sum_{i=1, j \neq i}^n \frac{|a_{ij}|}{|a_{ii}|} \leq 1, i = 1, 2, 3, \dots, n$$

where the inequality holds in case of at least one equation. Convergence is assured in the Gauss-Seidal method if the matrix A is diagonally dominant and positive definite. If it is not in a diagonally dominant form, it should be connected to a diagonally dominant form by row exchanger, before starting the Gauss-Seidal iterative scheme.

Gauss-Seidal method is also an iterative solution procedure which is an improved version of Jacobi's method. The method is also known as the method of successive approximations. Consider the system of linear simultaneous equations

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned} \tag{2.49}$$

If the absolute value of the largest coefficient in each equation is greater than the sum of the absolute values of all the remaining coefficients, then the Gauss-Seidal iteration method will converge. If this condition is not satisfied, then Gauss-Seidal method is not applicable. Here, in Eq.(2.54), we assume the coefficient a_{11} , a_{22} and a_{33} are the largest coefficients.

We can rewrite Eq.(2.54) as

$$\begin{aligned} x_1 &= \frac{1}{a_{11}}(b_1 - a_{12}x_2 - a_{13}x_3) \\ x_2 &= \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3) \\ x_3 &= \frac{1}{a_{33}}(b_3 - a_{31}x_1 - a_{32}x_2) \end{aligned} \tag{2.55}$$

Let the initial approximations be x_1^0 , x_2^0 and x_3^0 respectively. The following iterations are then carried out.

Iteration 1: The first improvements of x_1 , x_2 and x_3 are obtained as

$$\begin{aligned} x_{11} &= \frac{1}{a_{11}}(b_1 - a_{12}x_2^0 - a_{13}x_3^0) \\ x_2 &= \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3^0) \end{aligned}$$

$$x_3 = \frac{1}{a_{33}}(b_3 - a_{31}x_1 - a_{32}x_2) \quad (2.56)$$

Iteration 2: The second improvements of x_1 , x_2 and x_3 are obtained as

$$\begin{aligned} x_{11} &= \frac{1}{a_{11}}(b_1 - a_{12}x_{11} - a_{13}x_{31}) \\ x_2 &= \frac{1}{a_{22}}(b_2 - a_{21}x_{12} - a_{23}x_{31}) \\ x_3 &= \frac{1}{a_{33}}(b_3 - a_{31}x_{12} - a_{32}x_{22}) \end{aligned} \quad (2.56)$$

The above iteration process is continued until the values of x_1 , x_2 and x_3 are obtained to a pre-assigned or desired degree of accuracy. In general, the initial approximations are assumed as $x_1^0 = x_2^0 = x_3^0 = 0$. Gauss-Seidal method generally converges for any initial values of x_1^0 , x_2^0 and x_3^0 . The convergence rate of Gauss-Seidal method is found to be twice to that of Jacobi's method. Like the Jacobi's method, Gauss-Seidal method can also be extended to n linear simultaneous algebraic equations in n unknowns.

EXAMPLE:

Solve the following equations by Gauss-Seidal method.

$$\begin{aligned} 8x + 2y - 2z &= 8 \\ x - 8y + 3z &= -4 \\ 2x + y + 9z &= 12 \end{aligned}$$

Solution:

In the above equations:

$$\begin{aligned} |8| &> |2| + |-2| \\ |-8| &> |1| + |3| \\ |9| &> |2| + |1| \end{aligned}$$

So, the conditions of convergence are satisfied and we can apply Gauss-Seidal method. Then we rewrite the given equations as follows:

$$\begin{aligned}
x_1 &= \frac{1}{a_1}(d_1 - b_1y^0 - c_1z^0) \\
y_1 &= \frac{1}{b_2}(d_2 - a_2x_1 - c_2z^0) \\
z_1 &= \frac{1}{c_3}(d_3 - a_3x_1 - b_3y_1)
\end{aligned} \tag{2.56}$$

Let the initial approximations be:

$$x_1^0 = x_2^0 = x_3^0 = \mathbf{0}$$

Iteration 1:

$$\begin{aligned}
x_1 &= \frac{d_1}{a_1} = \frac{8}{8} = \mathbf{1.0} \\
y_1 &= \frac{1}{b_2}(d_2 - a_2x_1) = \frac{1}{-8}(-4 - 4 \times 1.0) = \mathbf{0.625} \\
z_1 &= \frac{1}{c_3}(d_3 - a_3x_1 - b_3y_1) = \frac{1}{9}(12 - 2) \\
&= \mathbf{2 \times 1.0 - 1 \times 0.625} = \mathbf{1.042}
\end{aligned}$$

Iteration 2:

$$\begin{aligned}
x_2 &= \frac{1}{a_1}(d_1 - b_2y_1 - c_1z_1) \\
&= \frac{1}{8}(8 - 2 \times 0.625 - (-2) \times 1.042) = \mathbf{1.104} \\
y_2 &= \frac{1}{b_2}(d_2 - a_2x_2 - c_2z_1) \\
&= \frac{1}{-8}(-4 - 1 \times 1.104 - 3 \times 1.042) = \mathbf{1.029} \\
z_2 &= \frac{1}{c_3}(d_3 - a_3x_2 - b_3y_2) \\
&= \frac{1}{9}(12 - 2 \times 1.104 - 1 \times 1.029) = \mathbf{0.974}
\end{aligned}$$

Iteration 2:

$$\begin{aligned}
x_2 &= \frac{1}{a_1}(d_1 - b_2y_1 - c_1z_1) \\
&= \frac{1}{8}(8 - 2 \times 0.625 - (-2) \times 1.042) = \mathbf{1.104} \\
y_2 &= \frac{1}{b_2}(d_2 - a_2x_2 - c_2z_1)
\end{aligned}$$

$$= \frac{1}{-8}(-4 - 1 \times 1.104 - 3 \times 1.042) = \mathbf{1.029}$$

$$\begin{aligned} z_2 &= \frac{1}{c_3}(d_3 - a_3x_2 - b_3y_2) \\ &= \frac{1}{9}(12 - 2 \times 1.104 - 1 \times 1.029) = \mathbf{0.974} \end{aligned}$$

Iteration 3:

$$\begin{aligned} x_3 &= \frac{1}{a_1}(d_1 - b_1y_2 - c_1z_2) \\ &= \frac{1}{8}(8 - 2 \times 1.029 - (-2) \times 0.974) = \mathbf{0.986} \\ y_3 &= \frac{1}{b_2}(d_2 - a_2x_3 - c_2z_2) \\ &= \frac{1}{-8}(-4 - 1 \times 0.986 - 3 \times 0.974) = \mathbf{0.989} \\ z_3 &= \frac{1}{c_3}(d_3 - a_3x_3 - b_3y_3) \\ &= \frac{1}{9}(12 - 2 \times 0.986 - 1 \times 0.989) = \mathbf{1.004} \end{aligned}$$

Iteration 4:

$$\begin{aligned} x_4 &= \frac{1}{8}(8 - 2 \times 0.989 - (-2) \times 1.004) = \mathbf{1.004} \\ y_4 &= \frac{1}{-8}(-4 - 1 \times 1.004 - 3 \times 1.004) = \mathbf{1.002} \\ z_4 &= \frac{1}{9}(12 - 2 \times 1.004 - 1 \times 1.002) = \mathbf{0.999} \end{aligned}$$

Iteration 5:

$$\begin{aligned} x_5 &= \frac{1}{8}(8 - 2 \times 1.002 - (-2) \times 0.999) = \mathbf{0.999} \\ y_5 &= \frac{1}{-8}(-4 - 1 \times 0.999 - 3 \times 0.009) = \mathbf{1.0} \\ z_5 &= \frac{1}{9}(12 - 2 \times 0.999 - 1 \times 1.0) = \mathbf{1.0} \end{aligned}$$

Iteration 6:

$$\begin{aligned} x_6 &= \frac{1}{8}(8 - 2 \times 1 - (-2) \times 1) = \mathbf{1.0} \\ y_6 &= \frac{1}{-8}(-4 - 1 \times 1.0 - 3 \times 1.0) = \mathbf{1.0} \end{aligned}$$

$$z_6 = \frac{1}{9}(12 - 2 \times 1.0 - 1 \times 1.0) = 1.0$$

Convergence Of Gauss Seidal Method

The convergence of the Gauss-Seidel method depends on the properties of the coefficient matrix of the linear system. Specifically, for convergence to occur:

Diagonal Dominance or Symmetry: The matrix should be diagonally dominant or symmetric positive definite. Diagonal dominance means that the absolute value of each diagonal element should be greater than the sum of the absolute values of the other elements in the corresponding row.

Irreducibility: The matrix should be irreducible, meaning that it cannot be transformed into a strictly upper or lower triangular form by reordering rows and columns.

Consistency: The system of linear equations should have a unique solution. If the system is inconsistent or ill-conditioned, convergence may not be guaranteed.

Smoothness of Coefficients: The smoother the coefficients of the equations, the faster the convergence. Rough or oscillatory coefficients may slow down convergence or even prevent it.

Initial Guess: A good initial guess can significantly speed up convergence. Ideally, the initial guess should be close to the actual solution.

2.1.8 SOR Is Better Than Other Methods

The Successive Over-Relaxation (SOR) method is a popular iterative technique used to solve linear systems of equations. Here are some reasons why it is considered advantageous over other methods in certain situations:

1. **Faster Convergence:** SOR often converges faster than other iterative methods, such as Jacobi or Gauss-Seidel, especially for large sparse linear systems. This is particularly true when the spectral radius of the iteration matrix is close to one.

2. **Tuning Parameter for Acceleration:** SOR introduces a relaxation parameter, which can be adjusted to accelerate convergence. Properly choosing this parameter can significantly speed up the convergence rate of the method.

3. **More Stable Convergence:** SOR can sometimes exhibit more stable convergence properties compared to other methods, especially when dealing with ill-conditioned or diagonally dominant systems.

4. **Parallelization Potential:** SOR can be parallelized efficiently, making it suitable for implementation on parallel computing architectures, which can further accelerate the solution process for large systems.

5. **Flexibility:** SOR allows for flexibility in the choice of relaxation parameter, which can be adapted during the iteration process to optimize convergence speed for different types of linear systems.

6. **Applicability to Elliptic Partial Differential Equations (PDEs):** SOR is particularly well-suited for solving elliptic PDEs, such as Laplace's equation and Poisson's equation, due to its efficient handling of the resulting sparse linear systems.

However, it's essential to note that the superiority of SOR over other methods depends on the specific characteristics of the linear system being solved. In some cases, other methods may be more suitable or efficient. Additionally, the performance of iterative methods can be influenced by factors such as matrix structure, conditioning, and sparsity. Therefore, it's often beneficial to experiment with different methods to determine the most effective approach for a particular problem.

Successive Over-Relaxation (SOR) method stands out among other iterative methods for solving linear systems of equations due to its superior convergence properties, especially when dealing with large, sparse matrices. Unlike traditional iterative methods like Jacobi or Gauss-Seidel, SOR incorporates an additional parameter, the relaxation factor, which accelerates convergence by allowing the algorithm to overshoot the solution. This overshooting effectively "jumps ahead" towards the solution, bypassing oscillations and significantly reducing the number of iterations required for convergence. Furthermore, SOR offers a good balance between computational efficiency and stability, making it particularly suitable for solving systems with ill-conditioned matrices or when precise control over convergence rate is desired.

Moreover, SOR exhibits remarkable versatility, as its performance can be fine-tuned by adjusting the relaxation factor to match the spectral radius of the iteration matrix. This adaptability allows for

efficient convergence across a wide range of problem domains, from fluid dynamics simulations to structural analysis in engineering applications. Additionally, SOR's simplicity and ease of implementation make it an attractive choice for practical problems where computational resources are limited or when rapid prototyping of iterative solvers is necessary. Overall, the combination of accelerated convergence, stability, and versatility positions SOR as a highly effective method for solving linear systems, outperforming many other iterative techniques in various scenarios.

Chapter 3

Successive Over Relaxation Method

Definition 3.0.1. Successive Over-Relaxation (SOR) is a numerical method used to solve linear systems of equations, particularly those arising from discretized partial differential equations (PDEs).

It's an iterative method that improves upon the Gauss-Seidel method by introducing an additional parameter called the relaxation factor, typically denoted by ω .

For any iterative method, in finding $x^{(k)}$ from $x^{(k-1)}$, we move a certain amount in a particular from $x^{(k-1)}$ to $x^{(k)}$. This direction from $x^{(k-1)}$ to $x^{(k)}$ is the vector from $x^{(k)} - x^{(k-1)}$ since $x^{(k)} = x^{(k-1)} + (x^{(k)} - x^{(k-1)})$

If we assume that the direction from $x^{(k-1)}$ to $x^{(k)}$ is taking us closer, but not all the way, to the true solution $\mathbf{x}$, then it would make sense to move in the same direction $x^{(k)} - x^{(k-1)}$, but farther along that direction. Here is how we derive the SOR method from the Gauss-Seidel method. First, notice that we can write the Gauss-Seidel equation as:

$$Dx^{(k)} = Lx^{(k)} - Ux^{(k-1)}$$

so that

$$x^{(k)} = D^{-1}[b - Lx^{(k)} - Ux^{(k-1)}]$$

We can subtract $x^{(k-1)}$ from both the sides to get

$$x^{(k)} - x^{(k-1)} = D^{-1}[b - Lx^{(k)} - Dx^{(k-1)} - Ux^{(k-1)}]$$

Now, think of this as the Gauss Seidel correction $(x^{(k)} - x^{(k-1)})_{GS}$.

As suggested above, it turns out that convergence $x^{(k)} \rightarrow x$ of the sequence of approximate solutions to the true solution is often faster if we go beyond the standard Gauss Seidel correction. The idea of the SOR method is to iterate,

$$x^{(k)} = x^{(k-1)} + \omega(x^{(k)} - x^{(k-1)})_{GS},$$

where, as we just found

$$(x^{(k)} - x^{(k-1)})_{GS} = D^{-1}[b - Lx^{(k)} - Dx^{(k-1)} - Ux^{(k-1)}]$$

and where generally $1 < \omega < 2$. Notice that if $\omega = 1$ then this is the Gauss Seidel method. Written out in detail, the SOR method is

$$x^{(k)} = x^{(k-1)} + \omega D^{-1}[b - Lx^{(k)} - Dx^{(k-1)} - Ux^{(k-1)}]$$

We can multiply both sides by matrix D and divide both sides by ω to rewrite this as

$$\frac{1}{\omega}Dx^{(k)} = \frac{1}{\omega}Dx^{(k-1)} + [b - Lx^{(k)} - Dx^{(k-1)} - Ux^{(k-1)}]$$

then collect the $x^{(k)}$ terms on the left hand side to get,

$$\begin{aligned} \left(L + \frac{1}{D}\right)x^{(k)} &= \frac{1}{\omega}Dx^{(k-1)} + [b - Dx^{(k-1)} - Ux^{(k-1)}] \\ &= \left(\frac{1}{\omega}D - D - U\right)x^{(k-1)} + b. \end{aligned}$$

When we solve for $x^{(k)}$ we get

$$x^{(k)} = \left(L + \frac{1}{D}\right)^{-1} \left[\left(\frac{1}{\omega}D - D - U\right)x^{(k-1)} + b\right].$$

Notice that the SOR Method is also of the form

$$x^{(k)} = B_{SOR}x^{(k-1)} + C$$

The iteration matrix B_{SOR} that determines convergence of the SOR method is

$$\left(L + \frac{1}{D}\right)^{-1} \left(\frac{1}{\omega}D - D - U\right)$$

so optimal convergence is achieved by choosing a value of ω that minimizes

$$\left\| \left(L + \frac{1}{D} \right)^{-1} \left(\frac{1}{\omega} D - D - U \right) \right\|.$$

Thus finally, this method introduces a parameter ω whose role is to minimize the spectral radius, the largest in modulus eigen value, of their iterative matrix. Successive over relaxation is the acceleration of Gauss Seidal method for a suitable choice of relaxation parameter ω . So the iterative formula for SOR method in component form according to Young [10] is given as

$$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{i < j} a_{ij}x_j^{(k-1)} - \sum_{i > j} a_{ij}x_j^{(k)} \right)$$

and SOR method in the matrix form is given as

$$\begin{aligned} x^{(k)} &= (D + \omega L)^{-1} [-\omega U + (1 - \omega D)] x^{(k-1)} + (D + \omega L)^{-1} \omega b x^{(k)} \\ &= B_{SOR} x^{(k-1)} + C \end{aligned}$$

Where

$$B_{SOR} = (D + \omega L)^{-1} [-\omega U + (1 - \omega D)]$$

is an iteration matrix, and $C = \omega(D + \omega L)^{-1}b$ is an iteration vector.

3.0.1 Convergence of SOR Method

Theorem: Let $A \in L(\mathbb{R})^n$ be symmetric, positive definite. Then the SOR method converges for $0 < \omega < 2$.

Proof. For a symmetric, positive definite A the decomposition becomes $A = D - L - L^t$ where the diagonal elements of D are strictly positive. Thus, in particular, B is nonsingular and so must be $D - \omega L$. For $0 < \omega < 2$ the matrices

$$B = \frac{1}{\omega}(D - \omega L), C = \frac{1}{\omega}[(1 - \omega)D + \omega D^t]$$

are well defined and real, and we can write

$$\begin{aligned} B_{SOR} &= B^{-1}C \\ &= B^{-1} \left[\frac{1}{\omega} D - D + L^t \right] \end{aligned}$$

$$\begin{aligned}
&= B^{-1} \left[\frac{1}{\omega} (D - \omega L) + L + L^t - D \right] \\
&= B^{-1} (B - A) \\
&= 1 - B^{-1} A
\end{aligned}$$

The following short calculation shows that

$$\begin{aligned}
A - B_{SOR}^t A B_{SOR} &= A - (1 - B^{-1} A)^t A (1 - B^{-1} A) \\
&= (B^{-1} A)^t A + A B^{-1} A - (B^{-1} A)^t A (B^{-1} A) \\
&= (B^{-1} A)^t [B^t + B - A] B^{-1} A, \tag{3.1}
\end{aligned}$$

where

$$\begin{aligned}
B^t + B - A &= \frac{2}{\omega} D - L - L^t - (D - L - L^t) \\
&= \frac{1}{\omega} (2 - \omega) D.
\end{aligned}$$

Since $0 < \omega < 2$ this proves that $B^t + B - A$ is symmetric positive definite, whence (3.1) implies the same for $A - B_{SOR}^t A B_{SOR}$. Let now $\lambda \in \mathbb{C}^1$ be any eigenvalue of B_{SOR} and $u \in \mathbb{C}^n, u \neq 0$ a corresponding eigenvector. Then

$$u^* A u > u^* B_{SOR}^t A B_{SOR} u = (\lambda u)^* A (\lambda u) = |\lambda|^2 u^* A u$$

So that $|\lambda|^2 < 1$ and hence also

$$\rho(B_{SOR}) < 1 \text{ as claimed.}$$

Theorem: If all diagonal elements of $A = L(\mathbb{C})^n$ are non zero, then the spectral radius of the SOR iteration matrix satisfies,

$$\rho(B_{SOR}) \geq |\omega - 1|. \tag{3.2}$$

Hence a necessary condition for the SOR method to converge is $|\omega - 1| < 1$ (for $\omega \in \mathbb{R}$ this condition becomes $\omega \in (0, 2)$).

Proof: Because L is strictly lower triangular we have

$$\det D^{-1} = \det (D + \omega L)^{-1} \text{ and}$$

$$\begin{aligned}
\det(B_{SOR}) &= (D + \omega L)^{-1} \det(1 - \omega)D + \omega U \\
&= \det((1 - \omega)I + \omega D - 1U) \\
&= \det((1 - \omega)I) \\
&= (1 - \omega)^n,
\end{aligned}$$

since $D^{-1}U$ is strictly upper triangular. Since $\det(B_{SOR})$ is the product of the eigenvalues of B_{SOR} it follows that $\rho(B_{SOR}) \geq \det(B_{SOR}) = (1 - \omega)^n$ and therefore that (3.2) holds. In the positive-definite case the necessary condition is also sufficient for convergence [3], as the following theorem shows.

Definition 3.0.2. Let the $N \times N$ matrix A be partitioned into the form

$$A = \begin{bmatrix} A_{1,1} & \cdots & A_{1,q} \\ \vdots & \ddots & \vdots \\ A_{q,1} & \cdots & A_{q,q} \end{bmatrix}$$

where $A_{i,j}$ is an $n_i \times n_j$ submatrix and $n_1 + \dots + n_q = N$. The $q \times q$ block matrix of A is said to have A-property if there exists two disjoint nonempty subsets S_R and S_B of $1, 2, \dots, q$ such that $S_R \cup S_B = 1, 2, \dots, q$ and such that if $A_{i,j} \neq 0$ and $i \neq j$ then $i \in S_R$ and $j \in S_B$ or $i \in S_R$

Theorem: If the matrix A enjoys the A-property and if B_J has real eigenvalues, then the SOR method converges for any choice of $x^{(0)}$ if

$$\rho(B_J) < 1 \text{ and } 0 < \omega < 2.$$

Moreover,

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - (\rho(B_J))^2}}$$

and the corresponding asymptotic convergence factor is

$$\rho(B_{opt}) = \frac{1 - \sqrt{1 - (\rho(B_J))^2}}{1 + \sqrt{1 - (\rho(B_J))^2}}$$

Theorem: Let $A = D + L + U$ be a matrix that satisfies the technical assumption

$$(3.3) \quad \det(Dk - \gamma L - \gamma - 1U) = \det(kD - L - U), \text{ for all } k, \gamma \in \mathbb{R} - 0$$

and B_J and B_{SOR} the iteration matrices of the Jacobi and SOR methods as defined above. If μ is an eigenvalue of b_j and $\lambda \neq 0$ satisfies,

$$\mu = \frac{\lambda + \omega - 1}{\omega \lambda^{1/2}} \quad (3.4)$$

for some $\omega \in (0, 2)$ then λ is an eigenvalue of B_{SOR} .

Proof: We have the definitions for B_J and B_{SOR} that

$$B_J = -D^{-1}(L + U), B_{SOR} = (D + \omega L)^{-1}[-\omega U + (1 - \omega)D].$$

Let λ be one eigenvalue of B_{SOR} , $\lambda \neq 0$. Because U and L are strictly upper and lower triangular matrices, and their main diagonals are zeros, we have

$$\begin{aligned} \det(-D^{-1})\det(D - \omega L) &= 1 \\ \det(B_{SOR} - \lambda I) &= \det[(D + \omega L)^{-1}(1 - \omega)D + \omega U - \lambda I] \\ &= \det(D^{-1})\det(D - \omega L)\det[(D + \omega L)^{-1}(1 - \omega)D + \omega U - \lambda I] \\ &= \det[(1 - \omega)I + \omega D - 1U - \lambda I + \omega \Lambda D - 1L] \\ &= \omega^n \det \left[\left(\frac{1}{\omega} - 1 \right) I + D^{-1}U - \frac{\Lambda}{\omega} + \lambda D^{-1}L \right] \\ &= (-1)^n \omega^n \lambda^{1/2} \det(D^{-1}) \det \left[\frac{(\omega + \lambda - I)}{(\omega \lambda^{1/2})} D - \lambda^{-1/2}U - \lambda^{1/2}L \right] \\ &= (-1)^n \omega^n \lambda^{1/2} \det \left[\frac{\omega + \lambda - I}{\omega \lambda^{1/2}} I - D^{-1}U - D^{-1}L \right] \\ &= (-1)^n \omega^n \det \left[B_J - \frac{\omega + \lambda - I}{\omega \lambda^{1/2}} I \right]. \end{aligned}$$

Because $\lambda \neq 0$ satisfies

$$\mu = \frac{\lambda + \omega - 1}{\omega \lambda^{1/2}},$$

where μ is an eigenvalue of B_J . In turn, when μ is an eigenvalue of B_J , and λ satisfies that relation, then λ is an eigenvalue of B_{SOR} .

Theorem: Assume $A = D + L + U$ is a matrix that satisfies (3.4), and assume that $B_J = I - D^{-1}A$ has only real eigenvalues and that $\beta = \rho(B_J) < 1$. Then the SOR iteration converges for every $\omega \in (0, 2)$, and

the spectral radius of the SOR matrix is

$$\rho(B_{SOR}) = f(\omega) = \frac{1}{4}[\omega\beta + \rho(\omega\beta)^2 - 4(\omega - 1)]^2, \text{ for } 0 < \omega \leq \omega_{opt}, (\omega - 1) > 0, \quad (3.5)$$

where ω_{opt} , the optimal value of ω is

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \beta^2}}. \quad (3.6)$$

For any other value of ω we have

$$\rho(B_{SOR_{opt}}) < \rho(B_{SOR}), \text{ for } \omega \in (0, 2) - \omega_{opt}.$$

Proof: For a given ω , $\rho(B_{SOR})$ is the largest eigenvalue of B_{SOR} in absolute value. Suppose that μ is an eigenvalue of B_J . From (3.4) we have

$$\lambda = \frac{1}{4} \left(\omega\mu \pm \sqrt{(\omega\mu)^2 - 4(\omega - 1)} \right)^2. \quad (3.7)$$

Equation (3.7) gives two eigenvalues for B_{SOR} . First, if

$$(\omega\mu)^2 - 4(\omega - 1) < 0,$$

then λ is imaginary, and the absolute value of λ is

$$|\lambda| = \left(\frac{1}{4}\omega^2\mu^2 + \omega - 1 - \frac{1}{4}\omega^2\mu^2 \right) = \omega - 1$$

$$\text{when } \omega^\sim \equiv \frac{(1 - \sqrt{1 - \mu^2})}{\mu^2} < \omega < 2.$$

Here, $\rho(B_{SOR}) = |\lambda|$ is independent of $\mu = \rho(B_J)$. Second, if $(\omega\mu)^2 - 4(\omega - 1) \geq 0$, then

$$\rho(B_{SOR}) = \max_{\mu \in \rho(B_J)} \frac{1}{4} \left[\omega|\mu| \pm \sqrt{(\omega\mu)^2 - 4(\omega - 1)} \right]^2,$$

$\omega \in \omega^\sim$. On one hand, for a fixed ω , $\rho(B_{SOR})$ is an increasing function with respect to variable $|\mu|$. Hence, to get the spectral radius of B_{SOR} ,

$$\text{let } |\mu| = \rho(B_J) = \beta$$

We have
$$\rho(B_{SOR}) = \frac{1}{4} \left[\omega|\mu| \pm (\omega\mu)^2 - 4(\omega - 1) \right]^2.$$

On the other hand, $\rho(B_{SOR})$ can be proved to be a decreasing function with respect of ω in $(0, \omega^\sim)$. Details are as follows: We have the first order derivative of $\rho(B_{SOR})$.

$$\rho'(B_{SOR}) = \frac{1}{2} \left(\omega\beta + \sqrt{(\omega\beta)^2 - 4(\omega - 1)\beta} + \frac{\beta^2 - \omega}{(\omega\beta)^2 - 4(\omega - 1)} \right).$$

To determine its sign, we need to examine the sign of

$$\left(\beta + \frac{\beta^2 - \omega}{(\omega\beta)^2 - 4(\omega - 1)} \right).$$

Because $\sqrt{(\omega\beta)^2 - 4(\omega - 1)}, \beta < 1, \omega < 2$

and
$$\begin{aligned} \beta\sqrt{(\omega\beta)^2 - 4(\omega - 1)} + (\beta^2\omega - 2) &< \sqrt{\omega^2 - 4\omega + 4} + \omega - 2 \\ &= 2 - \omega + \omega - 2 \\ &= 0. \end{aligned}$$

Therefore, $\rho'(B_{SOR}) < 0$ in the interval $(0, \omega^\sim)$, which implies that $\rho(B_{SOR})$ is a decreasing function of ω . When $\omega = \omega^\sim$, $\rho(B_{SOR})$ get its minimum in the interval $(0, \omega^\sim)$. We have proved above that in the interval $(\omega^\sim, \rho(B_{SOR}) = \omega - 1)$ is an increasing function and also gets its minimum when ω approaches to $\omega^\sim$. Moreover, $\rho(B_{SOR})$ is continuous at the point $\omega^\sim$. Considering that the optimal parameter ω is the very number that makes $\rho(B_{SOR})$ gets its minimum.

Therefore

$$\omega^\sim = \omega_{opt} = \frac{2(1 - \sqrt{1 - \beta^2})}{\beta^2} = \frac{2}{(1 + \sqrt{1 - \beta^2})}$$

According to the statement above, for any value of ω

$$\rho(B_{SORopt}) < \rho(B_{SOR}), \omega \in (0, 2) - \omega_{opt}$$

Example: Let us consider following system of equations,

$$2x + y = 1$$

$$x + 7y = 5$$

Solutions. Exact solution:

$$x = 0.153846153846154, y = 0.692307692307692.$$

Approximate solution:

Method: Jacobi method

No. of iterations: 15

Approximate solution: 0.153846157129925, 0.692307692516186

Error Vector: -0.328377147695846 , -0.020849333370876

Method: Gauss-Seidal method

No. of iterations: 8

Approximate solution: 0.153846157129925 , 0.692307691838582

Error Vector: -0.328377142144731, 0.046911019513374

Method: SOR method

No. of iterations: 6

Approximate solution: 0.153846158504998 , 0.692307691940860

Error Vector: -0.465884453237919 , 0.036683256432468

3.0.2 Advantages Of SOR

Successive Over-Relaxation (SOR) is an iterative method used to solve systems of linear equations, particularly those arising from the discretization of partial differential equations. Here are some advantages of the SOR method:

Faster Convergence: Compared to the basic Gauss-Seidel method, SOR often converges more rapidly. By introducing a relaxation factor, ω , the method accelerates the convergence rate, especially for large systems of equations.

Efficiency for Large Sparse Systems: SOR is particularly efficient for solving large, sparse systems of linear equations, which are common in engineering and scientific computations.

1. Control Over Convergence: The relaxation factor ω provides a way to control and optimize the convergence rate. Properly choosing ω can significantly improve the efficiency of the method.

2.Flexibility: SOR can be applied to both symmetric and non-symmetric matrices. This flexibility makes it a versatile tool in numerical linear algebra.

3.Simplicity of Implementation: The algorithm is relatively straightforward to implement, especially if one has already implemented the Gauss-Seidel method. The modification to include the relaxation factor is minimal.

4.Compatibility with Other Methods: SOR can be used in conjunction with other methods, such as preconditioning in iterative solvers, to enhance overall performance.

5.Memory Efficiency: Like other iterative methods, SOR requires relatively low memory compared to direct methods like Gaussian elimination, as it does not need to store large matrices or perform complex factorization.

6.Scalability: The method can be easily parallelized, which makes it suitable for high-performance computing applications where large-scale systems are solved.

7.Use in Partial Differential Equations: SOR is widely used in numerical solutions of partial differential equations, making it a critical tool in computational fluid dynamics, structural analysis, and other fields requiring the discretization of continuous problems.

In summary, the Successive Over-Relaxation method is a powerful and efficient technique for solving linear systems, particularly those that are large and sparse. Its main advantages lie in its faster convergence rate, efficiency, flexibility, and simplicity of implementation.

3.0.3 Disadvantages Of SOR

The Successive Over-Relaxation (SOR) method is an iterative technique used to solve linear systems, particularly those arising from the discretization of partial differential equations. While SOR can be more efficient than methods like Gauss-Seidel under certain conditions, it has several disadvantages:

1. Convergence Dependence on Relaxation Factor: - The

performance of SOR heavily relies on the choice of the relaxation factor (ω). If ω is not chosen appropriately, the method may converge very slowly or even diverge. Finding the optimal ω can be challenging and often requires trial and error or additional computational techniques.

2.Sensitivity to Initial Guess: - The convergence rate of SOR can be sensitive to the initial guess of the solution. A poor initial guess may lead to slow convergence or divergence.

3.Not Always Suitable for All Matrices: - SOR is particularly efficient for certain types of matrices (e.g., diagonally dominant or symmetric positive definite matrices). For other types of matrices, the method may perform poorly or fail to converge.

4.Computational Complexity: - For large systems, the computational overhead of SOR can be significant, especially if the optimal relaxation factor is not known beforehand. This overhead includes additional operations per iteration compared to simpler methods like the Gauss-Seidel method.

5.Implementation Complexity: - Implementing the SOR method can be more complex than simpler iterative methods. This complexity arises from the need to carefully manage the relaxation factor and ensure the stability of the iterations.

6.Sequential Nature: - SOR is inherently sequential, which can make it less efficient for parallel computing compared to other methods that are more naturally parallelizable.

7.Dependence on Matrix Properties: - The efficiency and effectiveness of the SOR method depend on specific properties of the matrix, such as its spectral radius. If the spectral radius is not well understood, predicting the behavior of the SOR method can be difficult.

8.Limited Applicability: - In some cases, the SOR method may not be applicable or effective, necessitating the use of other iterative or direct methods for solving the system of equations.

In summary, while the SOR method can be powerful and efficient under the right conditions, its disadvantages include sensitivity to parameters, initial conditions, and matrix properties, as well as challenges in implementation and computational complexity.

3.0.4 Types Of SOR

Successive Over-Relaxation (SOR) is an iterative method used to solve linear systems of equations, particularly those arising from discretized partial differential equations. Here are the different types or variations of the SOR method:

1. Standard SOR (Classical SOR): The traditional SOR method where each variable in the linear system is updated iteratively using a relaxation factor ω

2. Symmetric SOR (SSOR): An extension where the SOR method is applied in two sweeps: a forward sweep and a backward sweep. This method is particularly effective for symmetric positive definite matrices.

3. Line SOR (LSOR): This method updates entire lines or blocks of variables simultaneously instead of individual components. It is advantageous for problems with strong directional coupling, such as anisotropic diffusion problems.

4. Incomplete SOR (ISOR): A variation where the SOR method is applied to an incomplete version of the original matrix, often used to reduce computational effort.

5. Parallel SOR: Designed for parallel computing environments, this method involves parallelizing the updates of the variables to take advantage of multiple processors, often used in conjunction with domain decomposition techniques.

6. Gauss-Seidel with Over-Relaxation (GSOR): Sometimes referred to as SOR, but specifically highlights the Gauss-Seidel updating process with an over-relaxation factor.

Each type of SOR method is tailored to specific problem characteristics and computational environments, offering flexibility in handling various types of linear systems effectively.

3.1 Conclusion

In conclusion, the Successive Over-Relaxation (SOR) method represents a fundamental and effective approach to solving linear systems of equations, offering a balance between convergence speed and stability. By introducing a relaxation parameter, SOR accelerates the iterative process, enabling rapid refinement of the solution. This method's adaptability makes it well-suited for a wide range of applications, from solving elliptic partial differential equations to addressing large and sparse linear systems prevalent in scientific and engineering domains. Moreover, SOR's efficiency in exploiting sparsity and its compatibility with parallel computing architectures further enhance its appeal, particularly in the era of increasingly complex computational tasks.

Theoretical analyses underscore SOR's favorable convergence properties, especially in scenarios where other methods may struggle, such as diagonally dominant or well-conditioned systems. However, achieving optimal performance with SOR often requires careful selection of the relaxation parameter and consideration of system-specific characteristics. Despite these challenges, ongoing research continues to refine and extend SOR's capabilities, aiming to broaden its applicability and improve its robustness across various problem domains.

In practical terms, SOR stands as a cornerstone in the toolkit of numerical techniques, offering a versatile and efficient means of solving linear systems. Its widespread use in scientific simulations, engineering analyses, and computational modeling underscores its enduring relevance and utility in tackling real-world challenges. As computational demands evolve, the Successive Over-Relaxation method remains a reliable and indispensable tool for obtaining accurate solutions to complex linear systems efficiently.

We have concluded that the wrong choice of relaxation parameter may lead to poor convergence. Finding the value of the relaxation parameter is a very complex process for a very large system of equations. In particular, if coefficient matrix A is symmetric and positive definite, the SOR method converges more faster for $\omega_{opt} = \frac{2}{(1+\sqrt{1-\beta^2})}$. We have also concluded that, with the help of some typical examples, the SOR method converges more rapidly as compared to Jacobi and Gauss-Seidel methods by taking a carefully suitable choice of relaxation parameter $\omega \in (0, 2)$

References

- [1] F. Curtis, G. Patrick and O. Wheatley, Applied Numerical Analysis, 5th edition, Addison-Wesley, New York (1998) .
- [2] W. Hackbusch, Iterative Solution of Large Sparse Systems of Equations, Springer-Verlag, Heidelberg - New York (1994).
- [3] Numerical Analysis, NINTH EDITION, Richard L. Burden , Youngstown State University , J. Douglas Faires , Youngstown State University.
- [4] NUMERICAL METHODS by Rao V. Dukkipati Ph.D., P.E. ,Fellow of ASME and CSME Member Connecticut Academy of Sciences and Engineering.
- [5] *[https : //en.wikipedia.org/wiki/Successive_over – relaxation](https://en.wikipedia.org/wiki/Successive_over-relaxation)*
- [6] Study of Numerical Solution of linear system of equations by using SOR Algorithm with $0 < \omega < 2$.